

## Table of Contents

[AWG6GDefTopic](#)

[awg6G\\_about.1.2](#)

[awg6G\\_about.1.3](#)

[awg6G\\_about.1.4](#)

[awg6G\\_about.1.5](#)

[awg6G\\_about.1.6](#)

[awg6G\\_apps.5.01](#)

[awg6G\\_apps.5.02](#)

[awg6G\\_apps.5.03](#)

[awg6G\\_apps.5.04](#)

[awg6G\\_apps.5.05](#)

[awg6G\\_apps.5.06](#)

[awg6G\\_apps.5.07](#)

[awg6G\\_apps.5.08](#)

[awg6G\\_apps.5.09](#)

[awg6G\\_apps.5.10](#)

[awg6G\\_apps.5.11](#)

[awg6G\\_apps.5.12](#)

[awg6G\\_apps.5.13](#)

[awg6G\\_apps.5.14](#)

[awg6G\\_apps.5.15](#)

[awg6G\\_apps.5.16](#)

[awg6G\\_disp.4.1](#)

[awg6G\\_disp.4.2](#)

[awg6G\\_disp.4.3](#)

[awg6G\\_disp.4.4](#)

[awg6G\\_disp.4.5](#)

[awg6G\\_disp.4.6](#)

[awg6G\\_disp.4.7](#)

[awg6G\\_prog.3.01](#)  
[awg6G\\_prog.3.02](#)  
[awg6G\\_prog.3.03](#)  
[awg6G\\_prog.3.04](#)  
[awg6G\\_prog.3.05](#)  
[awg6G\\_prog.3.06](#)  
[awg6G\\_prog.3.07](#)  
[awg6G\\_prog.3.08](#)  
[awg6G\\_prog.3.09](#)  
[awg6G\\_prog.3.10](#)  
[awg6G\\_prog.3.11](#)  
[awg6G\\_prog.3.12](#)  
[awg6G\\_prog.3.13](#)  
[awg6G\\_prog.3.14](#)  
[awg6G\\_prog.3.15](#)  
[awg6G\\_prog.3.16](#)  
[awg6G\\_prog.3.17](#)  
[awg6G\\_prog.3.18](#)  
[awg6G\\_prog.3.19](#)  
[awg6G\\_prog.3.20](#)  
[awg6G\\_prog.3.21](#)  
[awg6G\\_prog.3.22](#)  
[awg6G\\_prog.3.23](#)  
[awg6G\\_theory.2.01](#)  
[awg6G\\_theory.2.02](#)  
[awg6G\\_theory.2.03](#)  
[awg6G\\_theory.2.04](#)  
[awg6G\\_theory.2.05](#)  
[awg6G\\_theory.2.06](#)  
[awg6G\\_theory.2.07](#)  
[awg6G\\_theory.2.08](#)  
[awg6G\\_theory.2.09](#)

- [awg6G\\_theory.2.10](#)
- [awg6G\\_theory.2.11](#)
- [awg6G\\_theory.2.12](#)
- [awg6G\\_theory.2.13](#)
- [awg6G\\_theory.2.14](#)
- [awg6G\\_theory.2.15](#)
- [awg6G\\_theory.2.16](#)
- [awg6G\\_theory.2.17](#)

# AWG6GDefTopic

<

>

## About the AWG6G and AWG6G Plus

### About the AWG6G and AWG6G Plus

The Arbitrary Waveform Generator 6G (AWG6G) is a high bandwidth analog source that creates complex, high frequency waveforms and fans out these waveforms to four differential outputs. Typical applications for the AWG6G include sourcing multitone signals for filter cutoff tests and simulating analog waveforms for testing disk drive read-channel and local area network (LAN) devices.

The AWG6G Plus includes all features of the AWG6G and extends the sample rate to 6.6GHz and the leveling frequency to 3GHz. In this description, the term "AWG6G" refers to the AWG6G and the AWG6G Plus unless otherwise noted.

The following topics are included:

- Features
- Safety Information

- [Key Terms](#)
- [AWG6G Software Licensing Requirements](#)
- [AWG6G Calibration](#)

AWG6G information includes the following sections:

- [AWG6G Theory of Operation](#)
- [AWG6G Programming](#)
- [AWG Debug Display](#)
- [AWG6G Examples](#)

Related Topics

- [AWG6G Specifications](#)
- [AWG6G Plus Specifications](#)
- [AWG \(VBT syntax\)](#)
- [AWG6G and AWG6G Plus DIB Design Guidelines](#)

# awg6G\_about.1.2

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## About the AWG6G and AWG6G Plus : Features

### Features

Standard features of the AWG6G include:

- Sampling frequency range of 400MHz to 6GHz for the AWG6G
- Sampling frequency range of 400MHz to 6.6GHz for the AWG6G Plus
- Filtered bandwidth of DC to 1350MHz
- Filtered bandwidth of DC to 2000MHz
- Unfiltered bandwidth of DC to 3000MHz for AWG6G Plus
- 10-bit waveform resolution
- Source memory depth of 16,711,680 (16M - 64K) samples
- Two AC amplitude ranges:
  - For frequencies up to 300MHz: 1.0V peak-to-peak
  - For frequencies greater than 300MHz: 632mV peak-to-peak

- [Maximum programmable DC offset of 2.0V](#)
- [Two peak output voltage ranges, AC and DC offset:](#)
    - [For frequencies up to 300MHz: +2.5V to -0.5V](#)
    - [For frequencies greater than 300MHz: +2.316V to -0.316V](#)
- [Four differential outputs \(eight identical channels\)](#)
- [Dual differential outputs through standard RF pipes](#)
- [Pin Parametric Measurement Units \(PPMUs\) behind each channel](#)

For hardware specifications, see [AWG6G Specifications](#) and [AWG6G Plus Specifications](#).

# awg6G\_about.1.3

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## About the AWG6G and AWG6G Plus : Safety Information

### Safety Information

When operating the AWG6G, always follow general UltraFLEX test system safety guidelines. See [Test System Safety](#).

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_about.1.4

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## About the AWG6G and AWG6G Plus : Key Terms

### Key Terms

|      |                                                                                    |
|------|------------------------------------------------------------------------------------|
| DDS  | Direct digital synthesis                                                           |
| FPGA | Field programmable gate array                                                      |
| ISL  | Instrument Sync-Link                                                               |
| LVDS | Low voltage differential signal, a protocol                                        |
| MSW  | Mixed Signal Workshop                                                              |
| PPMU | Pin Parametric Measurement Unit                                                    |
| PSet | Parameter set                                                                      |
| SMEM | Source Memory: Where the AWG6G stores the samples it uses to create waveforms      |
| TCIO | Tester computer I/O, a bus protocol that IG-XL uses to read and write HW registers |

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|



# awg6G\_about.1.5

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

About the [AWG6G](#) and [AWG6G Plus](#) : [AWG6G Software Licensing Requirements](#)

## AWG6G Software Licensing Requirements

The [AWG6G](#) and [AWG6G Plus](#) do not require a software license.  
For more information on [IG-XL licensing](#), see [IG-XL Licensing](#) in [IG-XL Administration help](#).

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_about.1.6

|                      |                      |
|----------------------|----------------------|
| <a href="#">&lt;</a> | <a href="#">&gt;</a> |
|----------------------|----------------------|

## About the AWG6G and AWG6G Plus : AWG6G Calibration

### AWG6G Calibration

Calibration for the AWG6G requires no user or program intervention or control. IG-XL automatic calibration manages all calibration factors. IG-XL automatically calibrates the AWG6G when necessary. It does this at least once every 90 days, or more frequently if the test head temperature varies by more than 3 degrees C.

The AWG6G does not require a calibration DIB.

To achieve amplitude accuracy at each programmed frequency, the AWG6G performs run-time calibration during a job run whenever you set a parameter that may affect calibration (such as level frequency, filter, sample rate, or amplitude). To prevent delays in your test program runs, IG-XL stores the results of run-time calibration and reuses them when you apply the same set of parameters.

Automatic calibration determines when to run levels calibration, and it uses a predefined calibration waveform to perform levels calibration. Autocal runs levels calibration using a sine wave at the frequency you set with the LevelFrequency parameter. You cannot turn off levels calibration. The range of the LevelFrequency parameter is from 200kHz to 2GHz for the AWG6G, 200kHz to 3GHz for the AWG6G Plus. If you set LevelFrequency greater than 2GHz on the AWG6G Plus, you must bypass the low-pass filter.

IG-XL determines whether to run level calibration whenever you:

- [Use the](#) Pins VBT language to change the value of Filter, Amplitude, SampleRate, ConnectionType, LoadImpedance, or LevelFrequency
- [Call the](#) Signals.LoadSettings method
- [Execute a PSet from within a pattern](#)

In these situations, IG-XL examines the combination of level frequency, sample rate, amplitude, and filter settings and looks for an existing levels calibration for these settings. If calibration results are available and are still valid, IG-XL applies the calibrated values to the hardware.

These calibration factors are also applied to PPMUs to ensure that the PPMUs provide the correct common mode voltage.

IG-XL also examines all PSets at the start of each pattern run and verifies that all have been calibrated. For any uncalibrated PSets, IG-XL performs levels calibration and stores the calibrated values in the PSet. After all PSets are calibrated, IG-XL allows the pattern to start.

Because IG-XL performs levels calibration only the first time it encounters a unique combination of settings or when the calibration expires, the first test program run can take more time than subsequent runs.

When programming using Pins language, changing any of the above parameters causes a levels calibration. In Signals programming, calibration runs when the settings are loaded.

For more information, see [Setup Calibration](#) and [Automatic Calibration](#).

<

>

2015 Teradyne, Inc. All rights reserved.

# awg6G\_apps.5.01

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Examples

### AWG6G Examples

This section contains the following kinds of program examples for the AWG6G:

- IG-XL workbook examples show all IG-XL data, VBT code, and patterns necessary for a complete running program.
- VBT function examples show one or more VBT functions that perform a programmed action when called as specified.

### Workbook Examples

- Source a Waveform and Perform PMU Measurements
- Source a Waveform from a File

### VBT Function Examples

- Create a Waveform Using the DSPWave.FileImport Method
- Start a Signal from VBT
- Start a Signal from a Digital Pattern

### Related Topics

- [AWG6G Programming](#)
- [AWG](#) (VBT Syntax Reference)

---

|                 |  |                                          |  |
|-----------------|--|------------------------------------------|--|
| <div>&lt;</div> |  | <div>&gt;</div>                          |  |
|                 |  | 2015 Teradyne, Inc. All rights reserved. |  |

# awg6G\_apps.5.02

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Examples : Source a Waveform and Perform PMU Measurements

### Source a Waveform and Perform PMU Measurements

This example sources a sine wave using VBT only. Mixed Signal Workshop determines timing. The configuration requires that the channels have the required termination for single-ended AWG channels (50ohms tied to ground). The AWG6G board is installed in slot 1. This workbook example includes the following program components:

- |   |                 |
|---|-----------------|
| • | DUT Connections |
| • | DataTool Sheets |
| • | VBT Code        |
| • | DSP Code        |

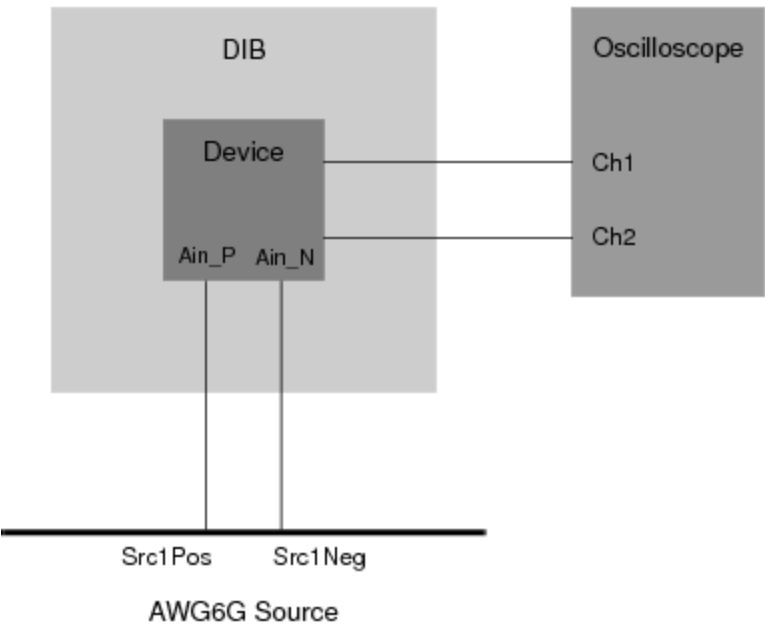
|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

# awg6G\_apps.5.03

|                                                                   |                                                                   |
|-------------------------------------------------------------------|-------------------------------------------------------------------|
| <input data-bbox="131 464 233 510" type="button" value=" &lt; "/> | <input data-bbox="829 464 932 510" type="button" value=" &gt; "/> |
|-------------------------------------------------------------------|-------------------------------------------------------------------|

AWG6G Examples : [Source a Waveform and Perform PMU Measurements](#) : DUT Connections

DUT Connections



AWG6G PPMU Connections

|                                                                     |                                                                     |
|---------------------------------------------------------------------|---------------------------------------------------------------------|
| <input data-bbox="131 1560 233 1606" type="button" value=" &lt; "/> | <input data-bbox="829 1560 932 1606" type="button" value=" &gt; "/> |
|---------------------------------------------------------------------|---------------------------------------------------------------------|

|                                          |  |  |
|------------------------------------------|--|--|
| 2015 Teradyne, Inc. All rights reserved. |  |  |
|------------------------------------------|--|--|

# awg6G\_apps.5.04

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Examples : Source a Waveform and Perform PMU Measurements : DataTool Sheets

### DataTool Sheets

This workbook example uses the following DataTool sheets:

- |   |                                           |
|---|-------------------------------------------|
| • | <a href="#">Pin Map Sheet</a>             |
| • | <a href="#">Channel Map Sheet</a>         |
| • | <a href="#">Test Instances Sheet</a>      |
| • | <a href="#">Flow Table Sheet</a>          |
| • | <a href="#">Global Specs Sheet</a>        |
| • | <a href="#">Mixed-Signal Timing Sheet</a> |
| • | <a href="#">Wave Definitions Sheet</a>    |

### Pin Map Sheet



## Pin Map

| Group Name | Pin Name | Type   | Comment                  |
|------------|----------|--------|--------------------------|
| AWG_Pins   | AWG_Pos1 | Analog | awg channel 1            |
|            | AWG_Neg1 | Analog | awg channel 2            |
|            | HSD_ch0  | I/O    | HSD channel 0            |
|            | AWG_Pos1 | Analog | awg ch 1 and 2 pin group |
|            | AWG_Neg1 |        |                          |

Channel Map Sheet

## Channel Map

DIB ID:

View Mode:

| Device Under Test |             | Tester Channel |           |          |
|-------------------|-------------|----------------|-----------|----------|
| Pin Name          | Package Pin | Type           | Site 0    | Comment  |
| AWG_Pos1          |             | AWGPos         | 1.srcpos1 | awg ch 1 |
| AWG_Neg1          |             | AWGNeg         | 1.srcneg1 | awg ch 2 |
| HSD_ch0           |             | I/O            | 16.ch0    | HSD ch 0 |

Test Instances Sheet

## Test Instances



| Test Procedure     |      |                 |           | DC Specs |          | AC Specs |          | Sheet Parameters |           |            |                     |         |
|--------------------|------|-----------------|-----------|----------|----------|----------|----------|------------------|-----------|------------|---------------------|---------|
| Test Name          | Type | Name            | Called As | Category | Selector | Category | Selector | Time Sets        | Edge Sets | Pin Levels | Mixed Signal Timing | Overlay |
| Source_Sine_Wave   | VBT  | Sine_Wave       |           |          |          |          |          |                  |           |            | SineWave            |         |
| continuity_awg_pis | VBT  | continuity_ppmu |           |          |          |          |          |                  |           |            |                     |         |
| Zin_FI             | VBT  | Zinput_FI       |           |          |          |          |          |                  |           |            |                     |         |
| Zin_FV             | VBT  | Zinput_FV       |           |          |          |          |          |                  |           |            |                     |         |

The test instance named continuity\_awg\_pis has the following additional test-specific parameters and values as shown in the Instance Editor:

| Instance: <input type="text" value="continuity_awg_pis"/> |                   |        |      |
|-----------------------------------------------------------|-------------------|--------|------|
| Parameters                                                | Timing and Levels | Limits |      |
| Name                                                      | Value             |        | Comm |
| ppmu_pins                                                 | AWG_Pins          | ...    |      |
| VSys                                                      | 0                 | ...    |      |
| Itest                                                     | -0.005            | ...    |      |

The test instance named Zin\_FI has the following additional test-specific parameters and values as shown in the Instance Editor:

Instance: Zin\_FI

Parameters

Timing and Levels

Limits

|                          | Name     | Value    |     |
|--------------------------|----------|----------|-----|
| <input type="checkbox"/> | ppmu_pin | AWG_Pins | ... |
| <input type="checkbox"/> | Itest    | 0.01     | ... |

The test instance named Zin\_FV has the following additional test-specific parameters and values as shown in the Instance Editor:

Instance: Zin\_FV

Parameters

Timing and Levels

Limits

|                          | Name     | Value    |     |
|--------------------------|----------|----------|-----|
| <input type="checkbox"/> | ppmu_pin | AWG_Pins | ... |
| <input type="checkbox"/> | Vtest    | 0.5      | ... |

Flow Table Sheet

Flow Table

TERADYNE

Flow Domain:

| Label | Enable | Gate |      |     | Command       |                    | TName | TNum | Limits |       | Datalog Display |       |        | Bin Number |      | Sort Number |      | Result |
|-------|--------|------|------|-----|---------------|--------------------|-------|------|--------|-------|-----------------|-------|--------|------------|------|-------------|------|--------|
|       |        | Job  | Part | Env | Opcode        | Parameter          |       |      | LoLim  | HiLim | Scale           | Units | Format | Pass       | Fail | Pass        | Fail |        |
|       |        |      |      |     | set-error-bin |                    |       |      |        |       |                 |       |        |            | 999  |             | 999  |        |
|       |        |      |      |     | Test          | continuity_awg_pis |       | 10   |        |       |                 |       |        |            |      |             |      |        |
|       |        |      |      |     | Use-Limit     | continuity_awg_pis |       |      | -0.5   | -0.1  |                 |       |        |            | 2    |             | 2    | Fail   |
|       |        |      |      |     | Test          | Zin_FI             |       | 20   |        |       |                 |       |        |            |      |             |      |        |
|       |        |      |      |     | Use-Limit     | Zin_FI             |       |      | 45     | 55    |                 |       |        |            | 4    |             | 4    | Fail   |
|       |        |      |      |     | Test          | Zin_FV             |       |      |        |       |                 |       |        |            |      |             |      |        |
|       |        |      |      |     | Use-Limit     | Zin_FV             |       | 30   | 45     | 55    |                 |       |        |            | 5    |             | 5    | Fail   |
|       |        |      |      |     | Test          | Source_Sine_Wave   |       | 110  |        |       |                 |       |        | 8          |      | 8           |      | None   |
|       |        |      |      |     | set-device    |                    |       |      |        |       |                 |       |        | 1          |      | 1           |      |        |

Global Specs Sheet

Global Specs

| Symbol      | Job | Value | Comment                     |
|-------------|-----|-------|-----------------------------|
| Vcl_default |     | -1    | Detector clamp voltage low  |
| Vch_default |     | 6     | Detector clamp voltage high |

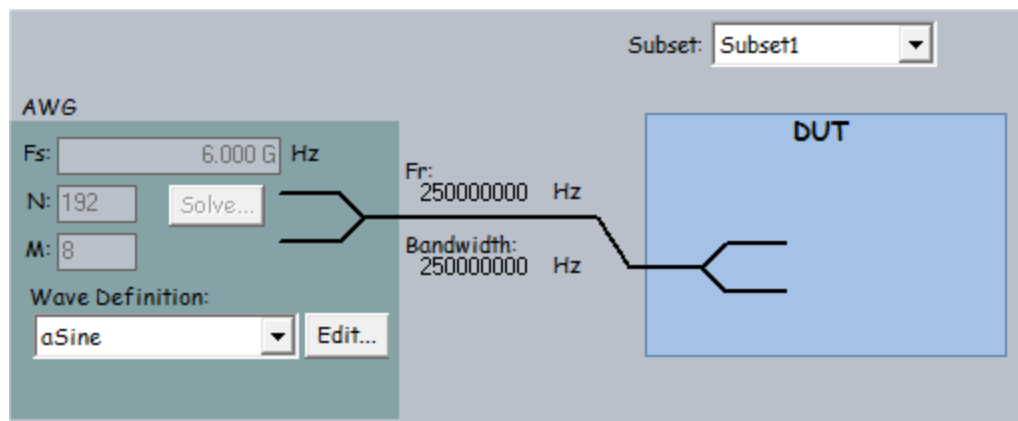
Mixed-Signal Timing Sheet

Mixed Signal Timing

TERADYNE

| Set Name | Subset  | Resource |    | Clocking   |     |           |   |     | Instrument | Waveform   |        | MSW      |
|----------|---------|----------|----|------------|-----|-----------|---|-----|------------|------------|--------|----------|
|          |         | Type     | ID | Fs         | N   | Fr        | M | USR | Data       | Definition | Filter | Settings |
| SineWave | Subset1 | AWG      |    | 6000000000 | 192 | 250000000 | 8 | 1   |            | aSine      |        |          |

This example uses the following Mixed Signal Workshop setup:



Mixed Signal Workshop's Solver provides the following timing solutions:

Solver

Select the item to be computed:

FS

N

=

FR

M

Sample Rate (FS)

Target: 6.000 GHz

Tolerance: +/- % (best FR with no limit on FS tolerance)

Repeat Frequency (FR)

Target: 250000000

Tolerance: +/- %

Sample Count (N)

192 to 4096

power of 2

multiple of 2

Number of Cycles (M)

1 to

odd

multiple of 1

Other Solution Criteria

☐ M,N relatively prime

Optimize for: 

FS

FR

| N   | M  | FS              | FS error | FR             | FR error        | Fres      | UTP (ms) | Rejected be |
|-----|----|-----------------|----------|----------------|-----------------|-----------|----------|-------------|
| 192 | 8  | 000,000,000.000 | 0.000    | 50,000,000.000 | 0.000           | 0,000.000 | 0.000    |             |
| 256 | 11 | 818,181,818.182 | 0.030    | 50,000,000.000 | 0.000           | 7,272.727 | 0.000    |             |
| 320 | 13 | 000,000,000.000 | 0.000    | 43,750,000.000 | 0.025           | 0,000.000 | 0.000    |             |
| 384 | 16 | 000,000,000.000 | 0.000    | 50,000,000.000 | 0.000           | 5,000.000 | 0.000    |             |
| 448 | 19 | 894,736,842.105 | 0.018    | 50,000,000.000 | 0.0000001192ppb | 7,894.737 | 0.000    |             |
| 512 | 21 | 000,000,000.000 | 0.000    | 46,093,750.000 | 0.016           | 8,750.000 | 0.000    |             |
| 576 | 24 | 000,000,000.000 | 0.000    | 50,000,000.000 | 0.000           | 6,666.667 | 0.000    |             |
| 640 | 27 | 925,925,925.926 | 0.012    | 50,000,000.000 | 0.0000002384ppb | 9,259.259 | 0.000    |             |
| 704 | 29 | 000,000,000.000 | 0.000    | 47,159,090.909 | 0.011           | 2,727.273 | 0.000    |             |
| 768 | 32 | 000,000,000.000 | 0.000    | 50,000,000.000 | 0.000           | 2,500.000 | 0.000    |             |

Show Choices

☐ Show all

62 solutions found  
62 evaluated

OK

Cancel

## Wave Definitions Sheet

## Wave Definitions



| WaveDefName | WaveDefType | WaveDef Component | Repeat Count | Relative Period | Relative Amplitude | Relative Offset | Primitive Parameters |
|-------------|-------------|-------------------|--------------|-----------------|--------------------|-----------------|----------------------|
| aSine       | Primitive   | Sine              | 1            | 1.E+00          | 1.E+00             | 0               | phase=90             |

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_apps.5.05

|   |   |
|---|---|
| < | > |
|---|---|

## AWG6G Examples : Source a Waveform and Perform PMU Measurements : VBT Code

### VBT Code

This example contains the following functions:

- **Function** Sine\_Wave associates the waveform created on the Wave Definitions sheet with a signal and sources the waveform.
- **Function** continuity\_ppmu performs a basic continuity test using the AWG PPMUs.
- **Functions** Zinput\_FI and Zinput\_FV measure resistance using voltage and current.

### Public Function Sine\_Wave() As Long

```
' First unload to clear memory
TheHdw.AWG("AWG_Pos1").Signals.Unload
TheHdw.AWG("AWG_Pos1").Signals.Add ("Sine")
With TheHdw.AWG("AWG_Pos1").Signals("Sine")
    .WaveDefinitionName = "aSine"
    .Amplitude = 0.25
    .CommonMode = 0#
    .Filter.LowPass.Bypass = False
    .ConnectionType = tlAWGConnectionTypeSingleEnded
    .LoadImpedance = 50
    .Calibration.LevelFrequency = 500000000
End With
' Connect positive channel of the board
TheHdw.AWG("AWG_Pos1").Connect tlAWGFromPOS, tlAWGToDUT
```

```

' Apply the values calculated in the MSW
TheHdw.AWG("AWG_Pos1").Signals("Sine").ApplyMixedSignalTiming
' Load the settings from the signal set onto hardware
TheHdw.AWG("AWG_Pos1").Signals("Sine").LoadSettings
' Start signal from VBT
TheHdw.AWG("AWG_Pos1").Signals("Sine").Start
TheHdw.AWG("AWG_Pos1").Signals.Stop
' Disconnect positive channel of the board
TheHdw.AWG("AWG_Pos1").Disconnect tIAWGFromPOS, tIAWGToDUT

```

End Function

```

Public Function continuity_ppmu(ppmu_pins As PinList, _
                               VSys As Double, _
                               Itest As Double) As Long
On Error GoTo errhandler
' Dimension object as PinListData to contain PPMU measured results
Dim pldPPMUMeasure As New PinListData
Dim IRng As Double
' Make sure AWG is disconnected.
' If not, you see the following error:
' AWGA:1026 : Attempt to connect path "ppmuchn1postodut" failed.
TheHdw.AWG("AWG_Pins").Disconnect tIAWGFromPOS, tIAWGToDUT
' Program PPMU Pins to force VsysValue. Connect the PPMUs and gate on.
With TheHdw.PPMU.Pins(ppmu_pins)
    .ForceV VSys
    .Connect
    .Gate = tIOn
    .ClampVHi = 1.5
    .ClampVLo = -1.2
End With
IRng = 2 * ma
If Abs(Itest) > 2 * ma Then IRng = 50 * ma
TheHdw.PPMU.Pins(ppmu_pins).Forcel Itest, IRng
' Make Measurements on PPMU pins and store in a PinListData
pldPPMUMeasure = TheHdw.PPMU.Pins(ppmu_pins).Read(tIPPMUReadMeasurements)
TheExec.Flow.TestLimit resultval:=pldPPMUMeasure, _
    ScaleType:=scaleMilli, _
    unit:=unitVolt, _

```

```

    forceVal:=Itest, _
    forcereults:=tlForceFlow

```

Exit Function ' No errors, so exit

errhandler:

If AbortTest Then Exit Function Else Resume Next

End Function

Public Function Zinput\_Fl(ppmu\_pin As PinList, Itest As Double) As Long

On Error GoTo errhandler

' Dimension objects as PinListData to contain PPMU measured

' and calculated results

Dim pldPPMUMeas As New PinListData

Dim pld\_outR As New PinListData

Dim IRng As Double

' Set up and Connect PPMU

With TheHdw.PPMU.Pins(ppmu\_pin)

.ForceV 0

.Connect

.Gate = tlOn

.ClampVHi = 4.5

.ClampVLo = -1.2

End With

TheHdw.Wait 100 \* us ' Allow settling (if needed)

' Decide current range

IRng = 2 \* ma

If Abs(Itest) > 2 \* ma Then IRng = 50 \* ma

' Force current and measure voltage

TheHdw.PPMU.Pins(ppmu\_pin).Forcel Itest, IRng

' Store measured Voltages in Pinlist Data

pldPPMUMeas = TheHdw.PPMU.Pins(ppmu\_pin).Read(tlPPMURedMeasurements)

' Do calculations using PinListData Math

pld\_outR = pldPPMUMeas.Math.Divide(Itest)

' Reset and disconnect PPMU

With TheHdw.PPMU.Pins(ppmu\_pin)

.ForceV 0

.Gate = tlOff

.Disconnect

End With

## ' Datalog results

```
TheExec.Flow.TestLimit resultval:=pld_outR, _
    ScaleType:=scaleNone, unit:=unitCustom, _
    customUnit:="Ohm", _
    forceVal:=Itest, _
    forcereults:=tlForceFlow
```

Exit Function

## errhandler:

If AbortTest Then Exit Function Else Resume Next

End Function

Public Function Zinput\_FV(ppmu\_pin As PinList, Vtest As Double) As Long

On Error GoTo errhandler

' Dimension objects as PinListData to contain PPMU measured

' and calculated results

Dim pldPPMUMeas As New PinListData

Dim pld\_outR As New PinListData

Dim IRng As Double

## ' Set up and Connect PPMUs

With TheHdw.PPMU.Pins(ppmu\_pin)

.ForceV 0

.Connect

.Gate = tlOn

.ClampVHi = 4.5

.ClampVLo = -1.2

End With

TheHdw.Wait 100 \* us ' Wait settling (if needed)

## ' Force voltage and measure current

TheHdw.PPMU.Pins(ppmu\_pin).ForceV Vtest

## ' Store measured current in PinListData

pldPPMUMeas = TheHdw.PPMU.Pins(ppmu\_pin).Read(tlPPMURadMeasurements)

## ' Do calculations using PinListData math

pld\_outR = pldPPMUMeas.Math.Divide(Vtest).Invert

## ' Reset and disconnect PPMUs

With TheHdw.PPMU.Pins(ppmu\_pin)

.ForceV 0

.Gate = tlOff

.Disconnect

End With



' [Datalog Results](#)

```
TheExec.Flow.TestLimit resultval:=pId_outR, _  
    ScaleType:=scaleNone, _  
    unit:=unitCustom, _  
    customUnit:="Ohm", _  
    forceVal:=Vtest, _  
    forcereults:=tIfForceFlow
```

[Exit Function](#)

[errhandler:](#)

If AbortTest Then Exit Function Else Resume Next

[End Function](#)

---

# awg6G\_apps.5.06

---

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Examples : Source a Waveform and Perform PMU Measurements : DSP Code](#)

---

## DSP Code

This example does not use DSP code.

---

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_apps.5.07

<

>

## AWG6G Examples : Source a Waveform from a File

### Source a Waveform from a File

This example sources a PRML waveform imported from an existing wave file. The example creates a signal using VBT values for Fs, M, N, and the wave definition, then sources the signal from an HSD pattern using pattern-oriented programming (POP) with PSets and patterns to synchronize and start the waveforms.

The configuration requires that the channels have the required termination for differential AWG channels (100 ohms tied to ground).

The AWG6G board is installed in slot 1.

This workbook example includes the following program components:

- DUT Connections
- DataTool Sheets
- WaveDesigner
- HSD Pattern with TSet and Pins
- VBT Code
- DSP Code

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

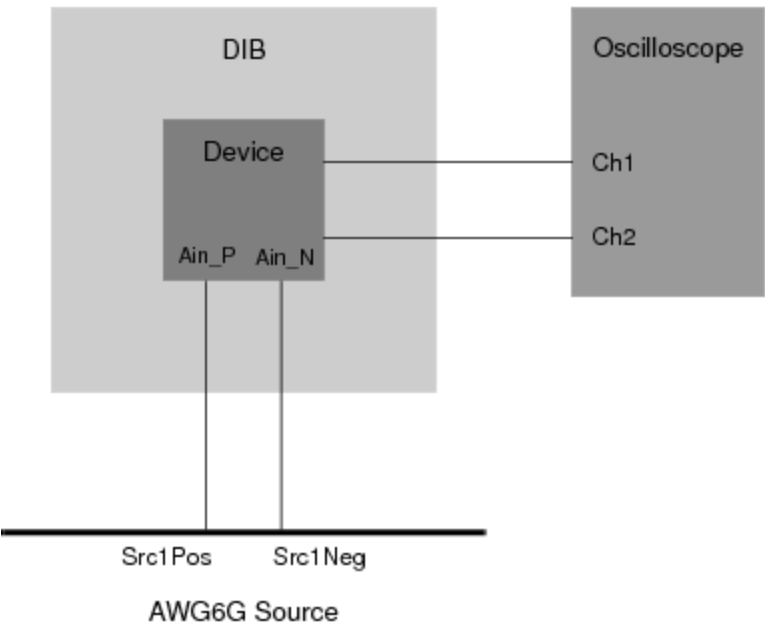
|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_apps.5.08

|                                                                   |                                                                   |
|-------------------------------------------------------------------|-------------------------------------------------------------------|
| <input data-bbox="131 464 233 510" type="button" value=" &lt; "/> | <input data-bbox="829 464 932 510" type="button" value=" &gt; "/> |
|-------------------------------------------------------------------|-------------------------------------------------------------------|

AWG6G Examples : [Source a Waveform from a File](#) : DUT Connections

DUT Connections



AWG6G Signal Connections

|                                                                     |                                                                     |
|---------------------------------------------------------------------|---------------------------------------------------------------------|
| <input data-bbox="131 1560 233 1606" type="button" value=" &lt; "/> | <input data-bbox="829 1560 932 1606" type="button" value=" &gt; "/> |
|---------------------------------------------------------------------|---------------------------------------------------------------------|

|                                          |  |  |
|------------------------------------------|--|--|
| 2015 Teradyne, Inc. All rights reserved. |  |  |
|------------------------------------------|--|--|

# awg6G\_apps.5.09

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Examples : Source a Waveform from a File : DataTool Sheets

### DataTool Sheets

This workbook example uses the following DataTool sheets:

- |   |                           |
|---|---------------------------|
| • | Pin Map Sheet             |
| • | Channel Map Sheet         |
| • | Time Sets (Basic) Sheet   |
| • | Pin Levels Sheet          |
| • | Test Instances Sheet      |
| • | Flow Table Sheet          |
| • | Global Specs Sheet        |
| • | Mixed-Signal Timing Sheet |
| • | Wave Definitions Sheet    |

### Pin Map Sheet

Pin Map

| Group Name | Pin Name | Type   | Comment            |
|------------|----------|--------|--------------------|
| AWG_Pins   | AWG_Pos1 | Analog | awg channel 1 pos  |
|            | AWG_Neg1 | Analog | awg channel 1 neg  |
|            | HSD_ch0  | I/O    | hsd channel        |
|            | AWG_Pos1 | Analog | awg ch 1 pin group |
| AWG_Pins   | AWG_Neg1 |        |                    |

Channel Map Sheet

Channel Map

DIB ID:

View Mode: Signal

| Device Under Test |             | Tester Channel |           |              |
|-------------------|-------------|----------------|-----------|--------------|
| Pin Name          | Package Pin | Type           | Site 0    | Comment      |
| AWG_Pos1          |             | AWGPos         | 1.srcpos1 | awg ch 1 pos |
| AWG_Neg1          |             | AWGNeg         | 1.srcneg1 | awg ch 1 neg |
| HSD_ch0           |             | I/O            | 16.ch0    | HSD ch 0     |

Time Sets (Basic) Sheet

Time Sets (Basic) 

Timing Mode: 

Single

Master Timeset Name: 

Tset\_1

Time Domain:

Strobe Ref Setup Name:

| Time Set | Cycle  |         | Pin/Group |       | Data |     | Drive |      |        |     | Compare |      |       |            | Edge Resolution |
|----------|--------|---------|-----------|-------|------|-----|-------|------|--------|-----|---------|------|-------|------------|-----------------|
|          | Period | Name    | Period    | Setup | Src  | Fmt | On    | Data | Return | Off | e       | Open | Close | Ref Offset | Mode            |
| Tset_1   | 1.E-06 | HSD_ch0 |           | i/o   | PAT  | NR  | 0     | 0    |        |     | Off     |      |       |            | Machine         |

Pin Levels Sheet

Pin Levels

| Pin/Group | Seq. | Parameter  | Value          |
|-----------|------|------------|----------------|
| HSD_ch0   |      | Vil        | 0              |
| HSD_ch0   |      | Vih        | 1              |
| HSD_ch0   |      | Vol        | 0              |
| HSD_ch0   |      | Voh        | 1              |
| HSD_ch0   |      | VCmpMain   | 0              |
| HSD_ch0   |      | Vt         | 0              |
| HSD_ch0   |      | Vcl        | =_Vcl_default  |
| HSD_ch0   |      | Vch        | =_Vch_default  |
| HSD_ch0   |      | VoutLoTyp  | 0              |
| HSD_ch0   |      | VoutHiTyp  | 1              |
| HSD_ch0   |      | DriverMode | Largeswing-HiZ |

Test Instances Sheet

Test Instances

TERADYNE

| Test Procedure   |      |           |           | DC Specs |          | AC Specs |          | Sheet Parameters |           |            |                     | Overlay |
|------------------|------|-----------|-----------|----------|----------|----------|----------|------------------|-----------|------------|---------------------|---------|
| Test Name        | Type | Name      | Called As | Category | Selector | Category | Selector | Time Sets        | Edge Sets | Pin Levels | Mixed Signal Timing |         |
| Source_PRML_Wave | VBT  | PRML_Wave |           |          |          |          |          | TSB              |           | Levels     |                     |         |

Source\_PRML\_Wave has the following string in the ArgList column:

AWGPins,AWGSignal,AWGPSet

Flow Table Sheet

Flow Table

TERADYNE

Flow Domain:

| Gate  |        |     |      |     | Command |                  |       |      | Limits |       | Datalog Display |       |        | Bin Number |      | Sort Number |      |        |
|-------|--------|-----|------|-----|---------|------------------|-------|------|--------|-------|-----------------|-------|--------|------------|------|-------------|------|--------|
| Label | Enable | Job | Part | Env | Opcode  | Parameter        | TName | TNum | LoLim  | HiLim | Scale           | Units | Format | Pass       | Fail | Pass        | Fail | Result |
|       |        |     |      |     | Test    | Source_PRML_Wave |       | 130  |        |       |                 |       |        | 1          | 8    | 1           | 8    |        |

Global Specs Sheet

Global Specs

| Symbol      | Job | Value    | Comment                     |
|-------------|-----|----------|-----------------------------|
| Vcl_default |     | - 1.E+00 | Detector clamp voltage low  |
| Vch_default |     | 6.E+00   | Detector clamp voltage high |

Mixed-Signal Timing Sheet

This sheet is not required because this example does not use mixed-signal timing.

Wave Definitions Sheet

This sheet is not required because this example uses an imported wave definition.

<

>

2015 Teradyne, Inc. All rights reserved.



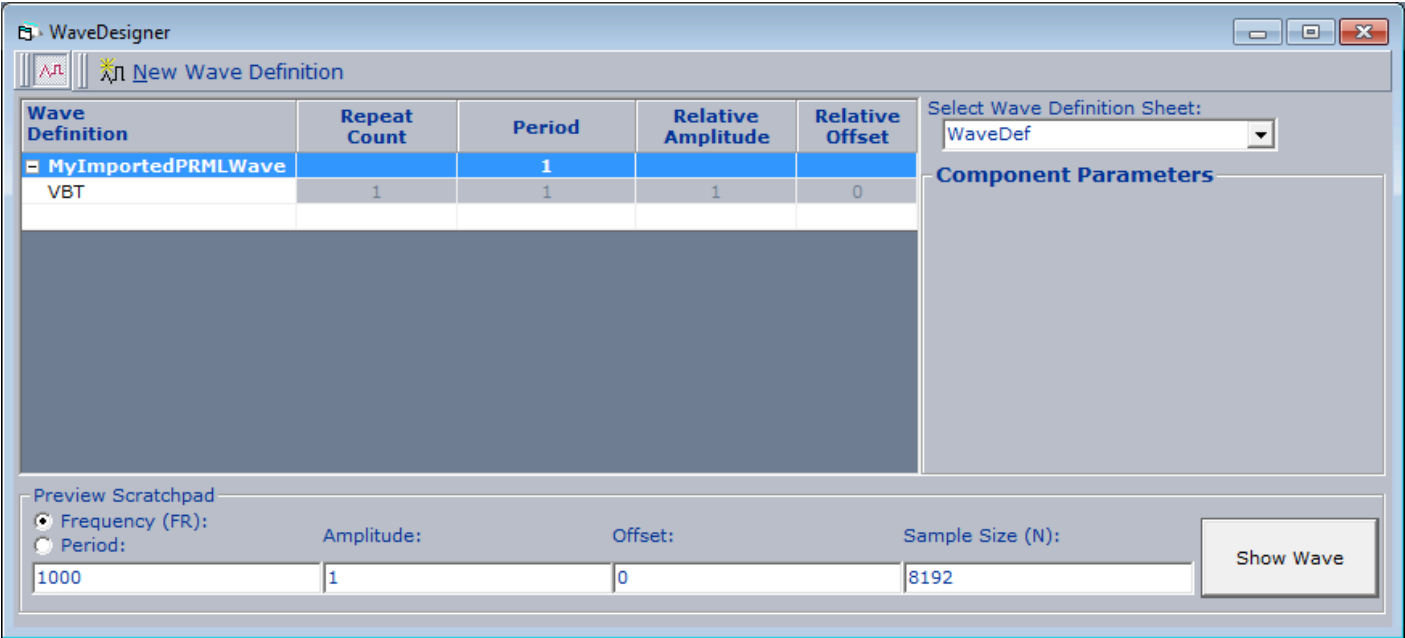
# awg6G\_apps.5.10

<

>

AWG6G Examples : Source a Waveform from a File : WaveDesigner

## WaveDesigner



<

>

# awg6G\_apps.5.11

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

## AWG6G Examples : Source a Waveform from a File : HSD Pattern with TSet and Pins

### HSD Pattern with TSet and Pins

The following pattern starts the AWG6G and sources the waveform.

To use this pattern, save the following text to a file named PRML\_Into\_Scope\_Pat\_Ctrl.atp, then import that file into PatternTool and compile it.

```
import tset TSet_1;
instruments = {
    AWG_Pins:AWG 1;
}
srm_vector
PRML_Into_Scope_Pat_Ctrl_module ( $tset, HSD_ch0)
{
> TSet_1 0    ;
start_label start_awg_prml_scope_view:
clr_cond_flags (cpuA_cond cpuB_cond cpuC_cond cpuD_cond) > TSet_1 0    ;
(AWG_Pins = PSet PRMLWave_PSet)
> TSet_1 0    ;
(AWG_Pins = Resync)
branch_expr = (!cpuA_cond) > TSet_1 0    ;
repeat 25000 > TSet_1 0    ;
(AWG_Pins = Start )
> TSet_1 0    ;
repeat 25000 > TSet_1 0    ;
global loop_prml:
> TSet_1 1    ;
> TSet_1 1    ;
```

[illegible]



>

# awg6G\_apps.5.12

<

>

AWG6G Examples : [Source a Waveform from a File](#) : VBT Code

VBT Code

This example contains the following functions:

- [Function](#) FileImportwNormalization imports the waveform from the specified file and normalizes the waveform.
- [Function](#) PRML\_wave associates the waveform with a signal, then starts a pattern that sources the waveform.

```
Public Function FileImportwNormalization(ByVal inWave As DSPWave, _
    ByVal NumSamples As Long, ByVal N As Double, _
    ByVal startingN As Double, ByVal M As Long, _
    ByVal Amplitude As Double, ByVal DCOffset As Double, _
    ByVal Ft As Double, ByVal FS As Double, _
    ParamArray userArgs()) As String
    Dim FileName As String
    Dim FileType As FileType
    Dim FileSampleRate As Double
    ' File name, type and Sample Rate are passed in as user
    ' arguments to the primitive
    FileName = userArgs(0)
    ' Import the data into inWave
    inWave.FileImport FileName, File_Image_wav
    inWave.Info.StorageLocation = "CPU"
```

```
' Convert the samples to doubles before normalization
```

```
inWave.Data = inWave.ConvertToReal.Data
```

```
' Normalize the waveform to +/-1
```

```
inWave.Data = inWave.DivideScalar(inWave.CalcMaximumMagnitude).Data
```

```
End Function
```

```
Public Function PRML_Wave() As Long
```

```
Dim pat_name As String
```

```
Dim pset_name As String
```

```
Dim signal_name As String
```

```
Dim pin_name As String
```

```
pat_name = "PRML_Into_Scope_Pat_Ctrl.pat"
```

```
pset_name = "PRMLWave_PSet"
```

```
signal_name = "PRMLWave_Signal"
```

```
pin_name = "AWG_Pins"
```

```
' Create PSets
```

```
TheHdw.AWG(pin_name).PSets.Add (pset_name)
```

```
With TheHdw.AWG(pin_name).PSets(pset_name)
```

```
    .Amplitude = 0.5
```

```
    .Calibration.LevelFrequency = 2000000000#
```

```
    .CommonMode = 0#
```

```
    .SampleRate = 5000000000#
```

```
End With
```

```
' Create Non-PSet Parameters for Waves
```

```
TheHdw.AWG(pin_name).Signals.Add (signal_name)
```

```
With TheHdw.AWG(pin_name).Signals(signal_name)
```

```
    .WaveDefinitionName = "MyImportedPRMLWave"
```

```
    .Amplitude = 0.5
```

```
    .CommonMode = 0#
```

```
    .Filter.LowPass.Bypass = False
```

```
    .Filter.LowPass = 2000000000#
```

```
    .Calibration.LevelFrequency = 2000000000
```

```
    .ConnectionType = tlAWGConnectionTypeDifferential
```

```
    .SampleRate = 5000000000#
```

```
    .SampleSize = 103552
```

```
    .LoadImpedance = 50
```

```
End With
```

```
' Connect both positive and negative channels of the board
```

```
' (You can use the debug display to confirm that this has happened)
```

```
TheHdw.AWG(pin_name).Connect tlAWGFromInst, tlAWGToDUT
' Load the signal settings onto AWG6G hardware
TheHdw.AWG(pin_name).Signals(signal_name).LoadSettings
TheHdw.AWG(pin_name).Signals.DefaultSignal = signal_name
Call TheHdw.Digital.ApplyLevelsTiming(True, True, True, tlPowered)
TheHdw.Patterns(pat_name).Load
TheHdw.Patterns(pat_name).Start "start_awg_prml_scope_view"
' Resume the pattern past loop by setting cpuA
Call TheHdw.Digital.Patgen.Continue(cpuA, 0)
' Debug routine to wait until pattern is halted
If TheExec.TesterMode = testModeOnline Then
    TheHdw.Digital.Patgen.HaltWait
Else
    TheHdw.Digital.Patgen.Halt
End If
' Disconnect both positive and negative channels of the board
' (You can use the debug display to confirm that this has happened)
TheHdw.AWG(pin_name).Disconnect tlAWGFromInst, tlAWGToDUT
```

End Function

---

|                                                                                     |                                                                                     |  |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--|
|  |  |  |
|                                                                                     | 2015 Teradyne, Inc. All rights reserved.                                            |  |

# awg6G\_apps.5.13

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

AWG6G Examples : [Source a Waveform from a File](#) : DSP Code

## DSP Code

This example does not use DSP code.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|



# awg6G\_apps.5.14

|   |   |
|---|---|
| < | > |
|---|---|

[AWG6G Examples](#) : Create a Waveform Using the DSPWave.FileImport Method

## Create a Waveform Using the DSPWave.FileImport Method

The following procedure adds rc\_48\_50\_pps1p6\_pw2p8.wav, a 19199-sample wave file which you created using IMAGE, to the Wave Definitions sheet.

1. [Place the following VBT function in any module:](#)

```
Function FileImportwNormalization(ByVal inWave As DSPWave, _
    ByVal NumSamples As Long, ByVal N As Double, _
    ByVal startingN As Double, ByVal M As Long, _
    ByVal Amplitude As Double, ByVal DCOffset As Double, _
    ByVal Ft As Double, ByVal FS As Double, _
    ParamArray userArgs()) As String
    Dim FileName As String
    Dim FileType As FileType
    Dim FileSampleRate As Double
    ' The File name, type and Sample Rate are passed in as user
    ' arguments to the primitive
    FileName = userArgs(0)
    ' Import the data into inWave
    inWave.FileImport FileName, File_Image_wav
    inWave.Info.StorageLocation = "CPU"
    ' Convert the samples to doubles before normalization
    inWave.Data = inWave.ConvertToReal.Data
    ' Normalize the waveform to +/-1
    inWave.Data = inWave.DivideScalar(inWave.CalcMaximumMagnitude).Data
End Function
```

2. Open WaveDesigner and add a waveform called MyImportedWave of type VBT primitive.

3. Fill in the WaveDesigner fields as follows:

– VBT routine: FileImportwNormalization

This is the name of the function you created earlier.

– Min samples: 19199

– Max samples: 19199

– Max Natural Envelope: 1

– Min Natural Envelope: -1

Setting the natural envelope allows IG-XL to skip the step of scanning the samples to determine the natural envelope and therefore saves time when the waveform is first loaded.

– Runtime Arg1: rc\_48\_50\_pps1p6\_pw2p8.wav

4. Save the definition and exit WaveDesigner.

The Wave Definitions sheet now includes the wave.

|                                                                                     |  |                                                                                     |  |
|-------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------|--|
|  |  |  |  |
| <div>2015 Teradyne, Inc. All rights reserved.</div>                                 |  |                                                                                     |  |

# awg6G\_apps.5.15

|                                                                           |                                                                           |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------|
| <input style="width: 100px; height: 30px;" type="button" value=" &lt; "/> | <input style="width: 100px; height: 30px;" type="button" value=" &gt; "/> |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------|

## AWG6G Examples : Start a Signal from VBT

### Start a Signal from VBT

The following function creates a new signal and assigns values to it, then starts the waveform using VBT.

**Public Function** VBT\_AWG6G\_Signal(PinSpec As String, \_  
WaveName As String, SegSize As Double, fsamp As Double) As Long

' Unload previous signals to make room for the current signal

TheHdw.AWG(PinSpec).Signals.Unload

' Add a signal to the collection for the specified pins

TheHdw.AWG(PinSpec).Signals.Add ("awg\_source")

' Assign values for the new signal

With TheHdw.AWG(PinSpec).Signals("awg\_source")

' Set amplitude to 0.3 V peak for the signal across 50 ohms

.Amplitude = 0.3

' Create a single-ended connection

.ConnectionType = tlAWGConnectionTypeSingleEnded

' Set the low-pass filter to 1.35 GHz

.Filter.LowPass = 135000000#

' Enable the low-pass filter

.Filter.LowPass.Bypass = False

' Set the load impedance for a single-ended connection

.LoadImpedance = 50

' Set the rate at which to sample the waveform

.samplerate = fsamp

' Set the sample size

.SampleSize = SegSize

```
' Assign the waveform to source
.WaveDefinitionName = wavename
' Perform most accurate calibration at the sampling rate
.Calibration.LevelFrequency = fsamp
' Provide a common mode voltage of 1.8V
.CommonMode = 1.8
End With
' Apply the signal settings
TheHdw.AWG(PinSpec).Signals("awg_source").LoadSettings
' Connect the AWG channels to the device
TheHdw.AWG(PinSpec).Connect tIAWGFromInst, tIAWGToDUT
' Start the signal
TheHdw.AWG(PinSpec).Signals("awg_source").Start
End Function
```

# awg6G\_apps.5.16

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

## AWG6G Examples : Start a Signal from a Digital Pattern

### Start a Signal from a Digital Pattern

This function uses Signals programming to define a waveform and a pattern to start a waveform on the AWG6G. Your pattern must include the following microcodes:

```
import_all_undefineds = yes;
import tset t0;
import svm_subr foo;
instruments = {
    AWG_CH1P:AWG 1;
    AWG_CH2P:AWG 1;
    AWG_CH3P:AWG 1;
    AWG_CH4P:AWG 1;
    AWG_CH5P:AWG 1;
    AWG_CH6P:AWG 1;
    AWG_CH7P:AWG 1;
    AWG_CH8P:AWG 1;
}
srm_vector      ( $tset, Ssource, Scope_Trigger, WS_input)
{
start_label begin:
loop0:
    > t0  1    1    X    ;
((AWG_CH1P, AWG_CH2P, AWG_CH3P, AWG_CH4P, AWG_CH5P, AWG_CH6P, AWG_CH7P,
AWG_CH8P) = Resync)
    > t0  1    1    X    ;
    > t0  1    1    X    ;
```

```

repeat 13541      > t0  1    1      X    ;
                  > t0  1    1      X    ;
((AWG_CH1P, AWG_CH2P, AWG_CH3P, AWG_CH4P, AWG_CH5P, AWG_CH6P, AWG_CH7P,
AWG_CH8P) = Start )
                  > t0  1    1      X    ;
                  > t0  1    1      X    ;
repeat 65001      > t0  1    1      X    ;
loop1:
                  > t0  1    1      X    ;
                  > t0  1    1      X    ;
                  > t0  1    1      X    ;
                  .
                  .
}

```

The following function sets up the HSD pattern, creates a new signal and assigns values to it, and starts the waveform from within the pattern.

```

Public Function AWG6G_Signal(PinSpec As String, PatName As String, _
    WaveName As String, SegSize As Double, fsamp As Double) As Long
' Start HSD Channel Setup
TheHdw.Digital.Pins("Source").InitState = chInitLo
TheHdw.Digital.Pins("Source").StartState = chStartLo
TheHdw.Digital.Pins("WS_input").InitState = chInitoff
TheHdw.Digital.Pins("WS_input").StartState = chStartOff
TheHdw.Digital.Pins("Scope_Trigger").InitState = chInitLo
TheHdw.Digital.Pins("Scope_Trigger").StartState = chStartLo
Call TheHdw.Patterns(PatName).Load
Call TheHdw.Digital.ApplyLevelsTiming(True, True, True, tlPowered)
' End HSD Channel Setup
' Unload previous signals to make room for the current signal
TheHdw.AWG(PinSpec).Signals.Unload
' Add a signal to the collection for the specified pins
TheHdw.AWG(PinSpec).Signals.Add ("awg_source")
' Assign values for the new signal
With TheHdw.AWG(PinSpec).Signals("awg_source")
    ' Set amplitude to 0.3 V peak for the differential signal across 100 ohms
    .Amplitude = 0.3
    ' Create a differential connection
    .ConnectionType = tlAWGConnectionTypeDifferential

```

```
' Set the low-pass filter to 2 GHz
.Filter.LowPass = 2000000000#
' Enable the low-pass filter
.Filter.LowPass.Bypass = False
' Set the load impedance for a differential connection
.LoadImpedance = 100
' Set the rate at which to sample the waveform
.samplerate = fsamp
' Set the sample size
.SampleSize = SegSize
' Assign the waveform to source
.WaveDefinitionName = wavename
' Perform most accurate calibration at the sampling rate
.Calibration.LevelFrequency = fsamp
' Provide a common mode voltage of 1.8V
.CommonMode = 1.8
```

End With

[' Apply the signal settings](#)

TheHdw.AWG(PinSpec).Signals("awg\_source").LoadSettings

[' Connect the AWG channels to the device](#)

TheHdw.AWG(PinSpec).Connect tIAWGFromInst, tIAWGToDUT

[' Start the pattern at the label "begin"](#)

[' This pattern must contain the microcodes given above](#)

Call TheHdw.Patterns(PatName).Start("begin", "")

[End Function](#)

# awg6G\_disp.4.1

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG Debug Display

### AWG Debug Display

The Arbitrary Waveform Generator debug display is an interactive diagram of the AWG hardware and signal path, which shows the instrument’s settings and connections in the context of a site. You can change the values, options, and settings in the display and run your test program to see the effects of the changes.

The following information is available:

- [AWG Debug Display](#)
- [Controlling the Source Signal](#)
- [Setting the Voltage Range, Amplitude, and Low-Pass Filter](#)
- [Viewing and Changing Connections](#)

**☑ Note:**

When a test program is running, it locks the test hardware for its exclusive use. For this reason, you must stop the test program to change a parameter. If the program is running (or waiting for input), you cannot change the display until the test program halts or is aborted.

This section describes features specific to the AWG debug displays. For more information on using debug displays, see the following:

- [Using Debug Displays in TDE](#)



|   |                                                    |
|---|----------------------------------------------------|
| • | <a href="#">Producing Code from Debug Displays</a> |
|---|----------------------------------------------------|

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_disp.4.2

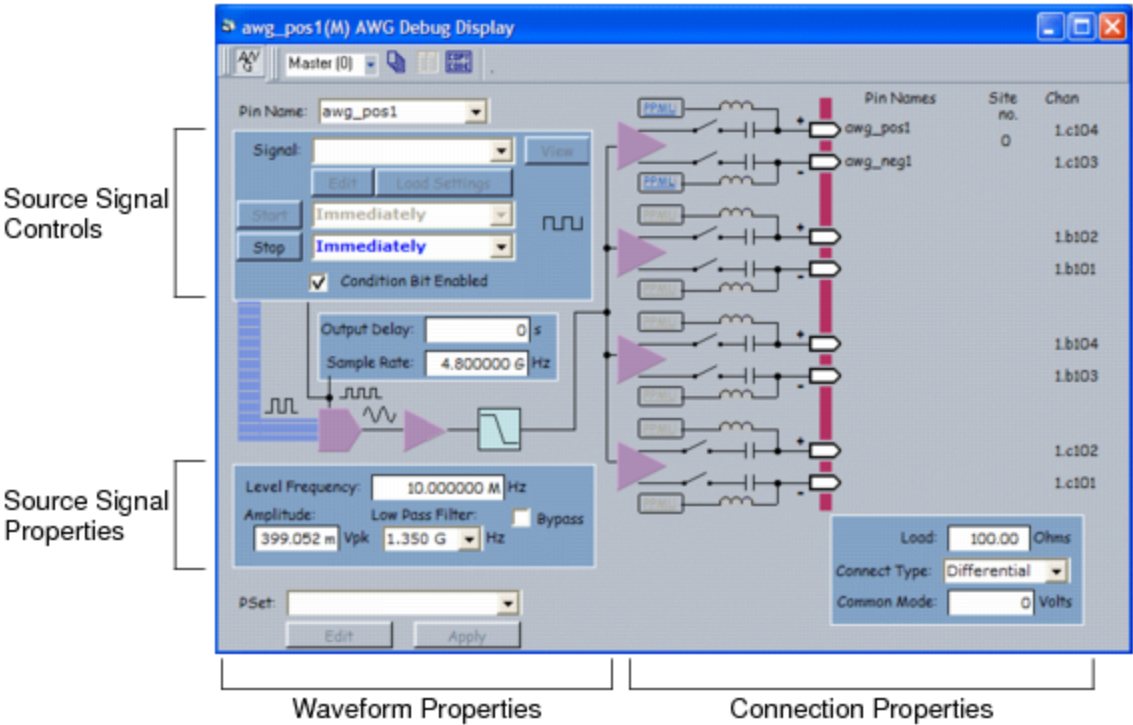
<

>

## AWG Debug Display : AWG Debug Display

### AWG Debug Display

The AWG Debug Display is an interactive display that shows a block diagram of the pin electronics, with the programmable parameters in text boxes. This display lets you view and change values for all AWG channels and provides channel-specific information.



### AWG Debug Display

<

>



# awg6G\_disp.4.3

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

AWG Debug Display : [AWG Debug Display](#) : Starting the AWG Debug Display

Starting the AWG Debug Display

To launch the AWG Debug Display, validate an AWG test program. Then, on the IG-XL Tools tab, in the Launch group, select Debug Displays>AWG. This opens the display for all active sites.

To display values, first specify the channel (pin plus site) and the time set (if needed) whose values you want. When you change the channel or time set, the new information is immediately displayed.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_disp.4.4

<

>

AWG Debug Display : [AWG Debug Display](#) : Viewing AWG Channel Information

Viewing AWG Channel Information

The display contains information only after you load and validate a test program. The following are the channel controls in the display:

AWG6G Channel Display Controls

| Control or Button | Description                                                                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Chan              | Shows the DIB channels to which the instrument connects for the current site.                                                                                                                                                                                        |
| Site no.          | Shows the site or sites to which each instrument connects. If an instrument connects to multiple sites, you can select a different site from the list.                                                                                                               |
| Pin Names         | Shows the pin or pins to which each instrument connects. If an instrument connects to multiple pins, you can select a different pin from the list.                                                                                                                   |
| PPMU              | <div></div> <p>Opens the PPMU display for the selected channel. If the test program does not contain a AWG pin for a specific channel, the link for that channel is disabled.</p> |

<

>

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_disp.4.5

<

>

## AWG Debug Display : Controlling the Source Signal

### Controlling the Source Signal

The items in the upper left section of the display include controls that select which signal to source and how the source instrument implements the start and stop commands. Note that most of the controls become active only when the Signal control contains a signal name.

The following table lists the parameters and buttons that control the sourcing of signals in the AWG debug display.

Source Waveform Debug Display Controls

| Control or Button | Description                                                                                                                                                                                                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pin Name          | Specifies the pin with which you want to work. The list contains all AWG pins. When you select one, the display shows you the entire board that contains that pin. If you select a pin on a different board, the display changes to show you the new board.                                                                                              |
| Signal            | Selects a signal from a list of defined signals. If you have defined a default signal, that signal’s name appears automatically in this field. If the menu contains no signal names, you have not defined any signals in VBT.                                                                                                                            |
| View              | <p>Opens the WaveScope tool and plots the sourced signal segment on the WaveScope display. This allows you to preview and to analyze the signal to source.</p> <p>When you run your test program on a test system, WaveScope displays the samples and scales them by the amplitude. When working offline, the WaveScope displays the ideal waveform.</p> |

|                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edit                                                                                | Brings up the PSet and Signals Editor (PSSE) with the selected signal loaded. This control is grayed out if no signals are available. See <a href="#">PSet and Signals Editor</a> in the TDE section.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Load Settings                                                                       | Loads the signal's timing settings as specified on the Mixed Signal Timing sheet, as well as all of the instrument's settings (such as voltage range, amplitude, and offset).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| Start                                                                               | <p>Starts sourcing the signal associated to the specified signal according to the action in the menu next to the Start button:</p> <ul style="list-style-type: none"> <li>• <a href="#">At End</a>: Starts sourcing the Signal at the end of the currently sourcing Signal and continues to loop the Signal.</li> <li>• <a href="#">Immediately</a>: Starts sourcing the Signal immediately and continues to loop the Signal. Default.</li> <li>• <a href="#">Once</a>: Starts sourcing the Signal immediately and stops when one cycle is complete.</li> <li>• <a href="#">Once at End</a>: Starts sourcing the Signal at the end of the currently sourcing Signal and stops when one cycle is complete.</li> </ul> |
| Stop                                                                                | <p>Stops sourcing according to the action in the menu next to the Stop button:</p> <ul style="list-style-type: none"> <li>• <a href="#">Immediately</a>: Stops sourcing the currently running Signal immediately. Does not affect a Signal that is running once. Default.</li> <li>• <a href="#">At End</a>: Stops sourcing the currently running Signal at the end.</li> </ul>                                                                                                                                                                                                                                                                                                                                      |
|  | The square wave moves when the AWG is sourcing a waveform. This feature works on a tester only.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Output Delay                                                                        | Sets the amount of time to pause before executing any microcode for the AWG.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Sample Rate                                                                         | Represents the number of samples to source per second.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |



Minimum sample rate = 400.000000MHz  
Maximum sample rate, AWG6G = 6.000000GHz  
Maximum sample rate, AWG6G Plus = 6.600000GHz

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_disp.4.6

<

>

## AWG Debug Display : Setting the Voltage Range, Amplitude, and Low-Pass Filter

### Setting the Voltage Range, Amplitude, and Low-Pass Filter

The items in the lower left section of the display include controls that determine the properties of the signal you want to source.

The following table lists the parameters and buttons that control the sourcing of signals in the AWG debug display.

| Source Waveform Properties Debug Display Controls |                                                                                                                                                                                                          |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Control or Button                                 | Description                                                                                                                                                                                              |
| Level Frequency                                   | See Setting the Leveling Frequency.                                                                                                                                                                      |
| Amplitude                                         | Sets the amplitude of the signal at the device.<br>Minimum amplitude: 25.179mV<br>Maximum amplitude: 1.002V                                                                                              |
| Low Pass Filter                                   | Selects the low-pass filter value. Valid values are 1.350GHz and 2.000GHz. Your selection must be valid for the value in Level Frequency.<br>Specifying a value out of range clears the Bypass checkbox. |
| Bypass                                            | Disables the low-pass filter and ignores the filter value.                                                                                                                                               |
| PSet                                              | Selects a parameter set (PSet). Selecting a PSet from the list does not apply the settings to hardware. To apply the PSet to hardware, click Apply.                                                      |

|       |                                                                                                                                                                                                      |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Edit  | Brings up the PSet and Signals Editor (PSSE) with the selected PSet loaded. This control is grayed out if no PSets are available.<br>See <a href="#">PSet and Signals Editor</a> in the TDE section. |
| Apply | Applies the PSet specified in the PSet field to the sourced signal. This control is grayed out if no PSets are available.                                                                            |

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_disp.4.7

<

>

## AWG Debug Display : Viewing and Changing Connections

### Viewing and Changing Connections

You can use the AWG debug display to set connection properties and to close or open relays to make or break connections.

### Setting Connection Properties

The items in the lower right section of the display include controls that determine the properties of the signal you want to source.

The following table lists the parameters and buttons that control the sourcing of signals in the AWG debug display.

AWG Debug Display Connection Properties

| Parameter    | Description                                                                                                         |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| Load         | Defines the load on the channel.<br>Minimum resistance: 5.00ohms<br>Maximum resistance: 1.00Mohms                   |
| Connect Type | Specifies single-ended or differential connection.                                                                  |
| Common Mode  | Sets the common mode voltage around which to center the AC signal.<br>Minimum Voltage: 0V<br>Maximum Voltage: 2.00V |

### Making Channel Connections

To open or close a relay, click the relay. The relay changes state and appears in bold.

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_prog.3.01

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming

### AWG6G Programming

This section describes programming of an AWG6G to source a waveform to a device.  
The following topics are available:

- Programming Concepts
- Programming Overview
- AWG6G DataTool Programming
- Creating a Wave Definition and Mixed Signal Timing Set
- AWG6G Pattern Control
- Using AWG6G in Concurrent Testing

For more information, see:

- AWG (VBT Syntax Reference)
- AWG6G Examples

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_prog.3.02

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Programming Concepts

### Programming Concepts

This section describes the following AWG6G programming concepts:

- |   |                                                                          |
|---|--------------------------------------------------------------------------|
| • | <a href="#">Simulated Configuration Files</a>                            |
| • | <a href="#">Common Mode Voltages</a>                                     |
| • | <a href="#">Controlling Waveform Amplitude Using Signals Programming</a> |
| • | <a href="#">Setting the Leveling Frequency</a>                           |
| • | <a href="#">Condition Bit Programming</a>                                |
| • | <a href="#">AWG6G Alarms and Error Reporting</a>                         |
| • | <a href="#">AWG6G Connections</a>                                        |
| • | <a href="#">AWG6G Reset Values</a>                                       |

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|



|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.03

<

>

## AWG6G Programming : Programming Concepts : Simulated Configuration Files

### Simulated Configuration Files

You can use the AWG6G offline by forcing IG-XL to map the AWG6G. Do this by creating a SimulatedConfig.txt file and placing the file in any of the following locations:

- Folder containing the IG-XL workbook
- \tester folder: C:\Program Files (x86)Teradyne\IG-XL\<version>\tester
- \bin folder: C:\Program Files (x86)\Teradyne\IG-XL\<version>\bin

The following SimulatedConfig.txt file maps the AWG6G board in slot 11:

```
<Version>
1
</Version>

<TEST_HEAD TH 1>
#slot[.subslot] Type          idprom (type serial rev company)
-1.0    dib                  239-001-00 0000000 0000-A 0000
1.0     HSD-M                979-219-00 0000000 0000-A 0000
        HSD-M                956-361-00 0000000 0000-A 0000
6.0     DC-30                805-002-00 0000000 0000-A 0000
        DC-30                949-901-00 0000000 0000-A 0000
11.0    AWG6G                805-008-01 0000000 0000-A 0000
        AWG6G                956-445-01 0000000 0000-A 0000
```

```
12.0    BBAC-15      979-214-00 0000000 0000-A 0000
        BBAC-15      956-375-00 0000000 0000-A 0000
24.0    SupportBoard 974-220-12 0000000 0000-A 0000
        SupportBoard 956-354-00 0000000 0000-A 0000
</TEST_HEAD TH 1>
```

For the AWG6G Plus, replace the information for slot 11 with the following:

```
11.0    AWG6G      607-976-00 0000000 0000-A 0000
        AWG6G      956-445-01 0000000 0000-A 0000
```

 **Note:**

The AWG6G and the AWG6G Plus both use the “AWG6G” board type. To differentiate between these boards, check the IDProm number on the first line of the entry. An IDProm of 805-008-01 specifies an AWG6G. An IDProm of 607-976-00 specifies an AWG6G Plus. This is true for CurrentConfig.txt that IG-XL generates and for SimulatedConfig.txt.

To use the AWG6G offline, the SimulatedConfig.txt file must include every board needed in the IG-XL workbook. The SimulatedConfig.txt file represents the state of the offline tester. To use the AWG6G online, you need not edit any files. IG-XL automatically maps all boards in the tester.

For more information, see [Tester Configuration](#).





2015 Teradyne, Inc. All rights reserved.

# awg6G\_prog.3.04

<

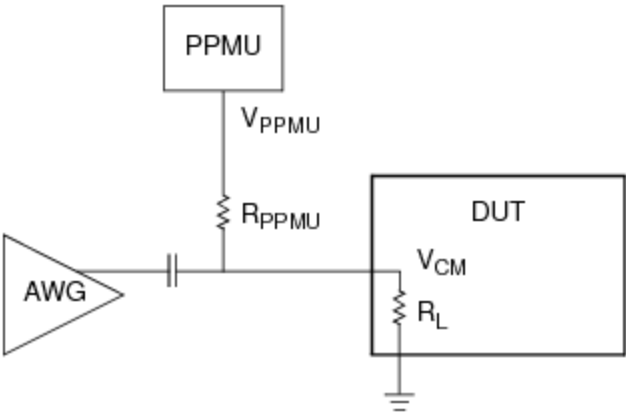
>

## AWG6G Programming : Programming Concepts : Common Mode Voltages

### Common Mode Voltages

Common mode voltage is a DC baseline voltage that appears at the pin of your device under test (DUT). The AWG6G uses the PPMU to create this voltage. You can specify a common mode voltage of from 0V to 2V.

The following figure shows a single-ended connection to the DUT. The AWG instrument is shown for reference and does not affect the common mode voltage.



### Single-Ended Connection to DUT where:

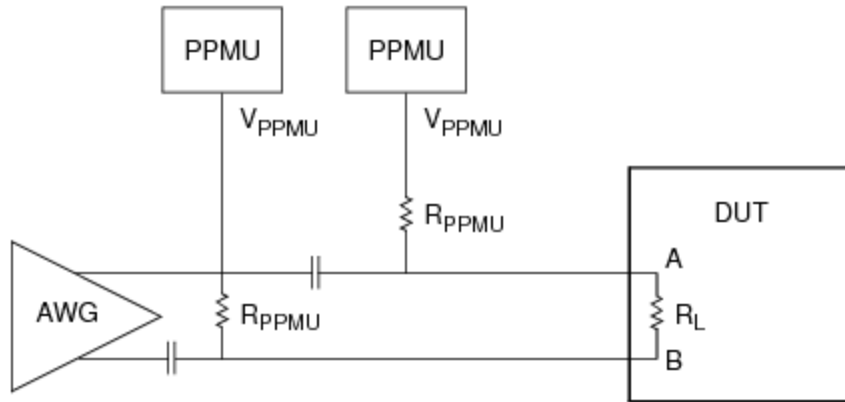
|            |                                                                  |
|------------|------------------------------------------------------------------|
| $R_{PPMU}$ | The resistance of the Bias-T circuitry associated with the PPMU. |
| $V_{PPMU}$ | The voltage forced by the PPMU.                                  |
| $R_L$      | The load impedance (programmable).                               |
| $V_{CM}$   | The expected voltage at the DUT, relative to $R_L$ .             |

IG-XL must compensate for a voltage drop across  $R_{PPMU}$ . IG-XL uses the following equation to calculate the required compensation (this equation is applicable for single-ended connections):

$$(1) \quad V_{PPMU} = V_{CM} \times \frac{R_{PPMU} + R_L}{R_L}$$

where  $V_{CM}$  is the common mode voltage relative to ground.

The following figure shows a differential connection to the DUT:



### Differential Connection

Because the DUT floats, there is no voltage drop across  $R_{PPMU}$ . Therefore,  $V_{PPMU} = V_{CM}$ .

The AWG6G uses the PPMUs to supply this common mode voltage. Therefore, you cannot connect both the PPMU and the AWG at the same time. Instead, you must disconnect one before you connect the other. IG-XL returns an error if you try to connect the AWG without first disconnecting the PPMU, or to connect the PPMU without first disconnecting the AWG.

Because you program the PPMU and AWG independently, IG-XL retains the common mode settings even if you perform PPMU. You can disconnect and reconnect one instrument without affecting the settings of the other. For example:

```
TheHdw.AWG.Pins("A_IN1").CommonMode = 1.25
```

```
TheHdw.AWG.Pins("A_IN1").Disconnect
```

```
TheHdw.PPMU.Pins("A_IN1").Connect
```

```
TheHdw.PPMU.Pins("A_IN1").ForceV 2
```

```
' Perform measurements using the PPMU
```

```
TheHdw.PPMU.Pins("A_IN1").Disconnect
```

```
TheHdw.AWG.Pins("A_IN1").Connect
```

```
Debug.Print TheHdw.AWG.Pins("A_IN1").CommonMode
```

Your output shows 1.25V.

<

>

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

## awg6G\_prog.3.05

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

### AWG6G Programming : Programming Concepts : Controlling Waveform Amplitude Using Signals Programming

#### Controlling Waveform Amplitude Using Signals Programming

When you create a waveform, IG-XL stores it in SMEM as a series of samples with values between 1 and -1. Before you source the signal, you program the amplitude as you want it to appear at the device. The AWG6G output stages then scale the waveform so that it appears at the device with the amplitude you specify.

When you use Signals programming, you set the amplitude for a signal, not the waveform. To change the amplitude of the waveform at the device, you must redefine your signal or use a different waveform.

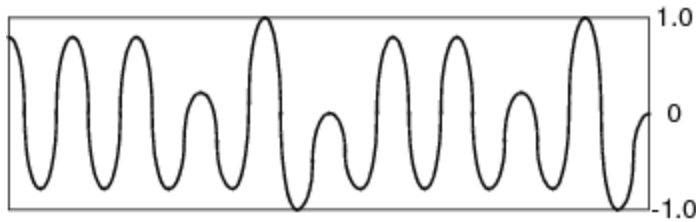
IG-XL provides a way for you to change the appearance of the waveform at the device without having to modify the signal definition. Using the `Signals.GainHardware` property, you can change the effect of the voltage control hardware between the AWG digital-to-analog converter and the device.

If you do not program a value for `GainHardware` or if you set the `GainHardware.Mode` property to `tlCSignalUseCalculatedValue`, IG-XL determines the value for this property, which is generally the same as the value you program for the `Signals.Amplitude` property.

If the values of the `Signals.GainHardware` and `Signals.Amplitude` properties differ (and `GainHardware.Mode` is either `tlCSignalUseValue` or `tlCSignalUseLoadedValue`), IG-XL calculates a scaling factor. This scaling factor is the value of the `Amplitude` property divided by the value of the `GainHardware` property. For example, if you set `Amplitude` to 0.2 and `GainHardware` to 0.8, the scaling factor is 0.25. IG-XL then loads a new set of samples to SMEM and scales the new waveform according to the calculated factor. The scaling is applied when you load the signal and takes the same time as a call to `LoadSamples`.

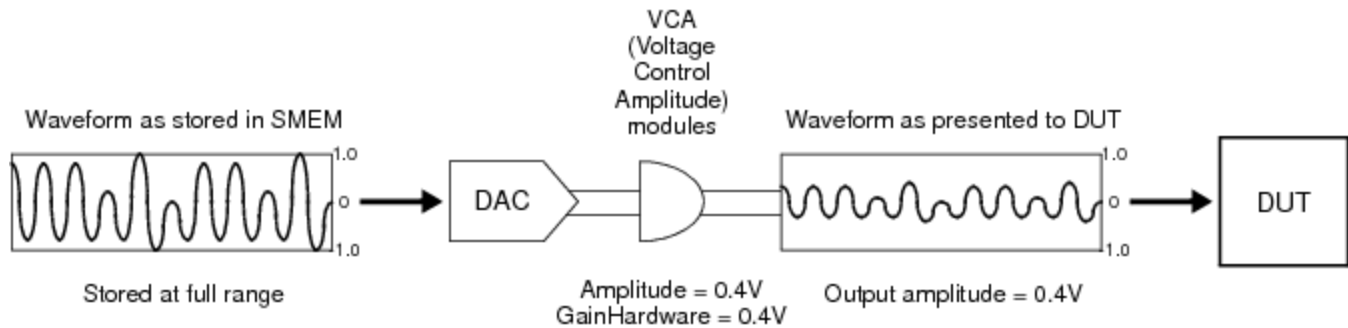
You cannot specify values for `Amplitude` and `GainHardware` that result in a scaling factor greater than 1, as this would produce a waveform that exceeds SMEM boundaries.

For example: You set up two signals, Sig1 and Sig2, to source the same waveform. For Sig1 you program the Signals.Amplitude property to 0.4; for Sig2 you program the Signals.Amplitude property to 0.2. For both signals you program the GainHardware property to 0.4 and set GainHardware.Mode to tlCSignalUseValue.



### Waveform as Stored in SMEM

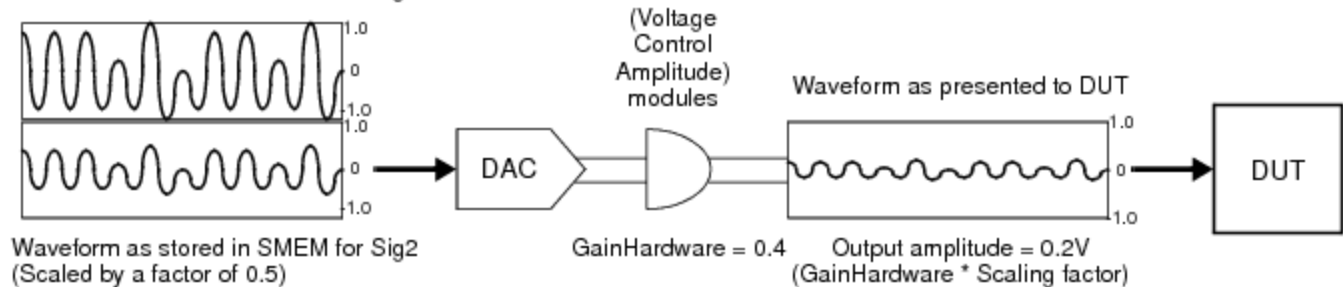
When you start Sig1, IG-XL loads and sources the signal with an amplitude of 0.4V.



### Sig1 as Sourced to Device

When you start Sig2, IG-XL calculates a scaling factor based on the relation of the gain hardware setting to the amplitude setting (in this example, the factor is 0.5) and loads a new set of samples to memory with this scale factor applied. IG-XL then sources the signal with an amplitude of 0.2V.

Waveform as stored in SMEM for Sig1



### Sig2 as Sourced to Device

The Signals.Amplitude property has no equivalent under the Pins language. When you program a value for Pins.Amplitude, you are programming the AWG6G output hardware in the same manner as when you specify Signals.GainHardware. Samples are not scaled, and IG-XL uses this value to set the amplitude of the waveform at the device.



|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.06

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

## AWG6G Programming : Programming Concepts : Setting the Leveling Frequency

### Setting the Leveling Frequency

IG-XL determines if it needs to run level calibration on the AWG6G whenever you:

- [Use the](#) Pins VBT language to change the value of Filter, Amplitude, SampleRate, ConnectionType, LoadImpedance, or LevelFrequency
- [Call the](#) Signals.LoadSettings method
- [Execute a PSet from within a pattern](#)

For more information, see [AWG6G Calibration](#).

The most efficient way to program settings for the AWG6G is to use the Signals properties or to create and call a PSet. Both of these methods check for levels calibration only once, when IG-XL applies the properties to the hardware.

### Calibration with Pins Language

[Because the](#) Pins language programs the AWG6G hardware directly, using the Pins language to configure the AWG6G causes IG-XL to check for levels calibration each time you set one of the above properties.

[For example:](#)

[With](#) TheHdw.AWG.Pins("AWG\_Pin")

- ' Set the leveling frequency; IG-XL checks for valid levels calibration  
.LevelFrequency = 200000000
- ' Set the filter frequency; IG-XL checks for valid levels calibration  
.Filter = 1350000000
- ' Set the sampling frequency; IG-XL checks for valid levels calibration

```
.SampleRate = 4000000000
```

```
' Set the waveform amplitude; IG-XL checks for valid levels calibration
```

```
.Amplitude = 2
```

```
End With
```

Your test program requires that the AWG6G be properly calibrated at the end of the With statement, after you have completed configuring the instrument. However, three of the four state changes are transitional states that do not affect the waveform you want to source. This requires spending time checking calibration for unused states (and, for the first run of the test program, performing calibration for each state).

### Calibration with Signals and PSet Programming

When you use a Signal to program the AWG6G, IG-XL applies all properties to the hardware at the same time, and therefore checks for calibration (and performs calibration if required) once. For example:

With TheHdw.AWG.Pins(myPin).Signals("SignalA")

```
' Set the leveling frequency in the signal definition
```

```
.LevelFrequency = 200000000
```

```
' Set the filter frequency in the signal definition
```

```
.Filter = 1600000000
```

```
' Set the sampling frequency in the signal definition
```

```
.SampleRate = 4000000000
```

```
' Set the waveform amplitude in the signal definition
```

```
.Amplitude = 2
```

```
' Apply all parameters at once; IG-XL checks for valid levels calibration
```

```
.LoadSettings
```

```
End With
```

IG-XL does not apply the settings to the hardware until the call to LoadSettings, and performs the calibration check at that time.

The same is true for PSet programming. When you define the PSet using VBT, IG-XL stores the settings but does not apply them to the hardware. The settings take effect, and IG-XL checks for levels calibration, only when a pattern executes the PSet.



## awg6G\_prog.3.07



### AWG6G Programming : Programming Concepts : Condition Bit Programming

#### Condition Bit Programming

The AWG6G provides a condition bit that you can use to control branching in a digital pattern. Using the [Enable\\_Inst\\_Cond](#) and [Disable\\_Inst\\_Cond](#) microcodes, you can open and close a window that allows the instrument condition pulse to move to the pattern generator. This allows you to select dynamically which instrument delivers a condition bit during a pattern burst. The window must be open when the instrument is ready to set the bit for the pulse to reach the pattern generator.

In addition to pattern microcode control of the AWG6G condition bit, you use the following VBT code to control enabling and disabling the bit on the channel:

' Enable the source instrument condition bit:

```
TheHdw.AWG.Pins(SrcPin).ConditionBitEnabled = True
```

' Disable the source instrument condition bit:

```
TheHdw.AWG.Pins(SrcPin).ConditionBitEnabled = False
```

You can also use the [ConditionData](#) VBT property to determine whether the AWG6G sets the condition bit at the beginning or end of the waveform.

For the AWG6G to set the condition bit, you must do both of the following:

- Use the [ConditionBitEnabled](#) VBT statement to enable the bit on that channel
- Use [Enable\\_Inst\\_Cond](#) microcode to enable the window

IG-XL OR's all UltraFLEX test system instrument condition bits before sending them to the pattern generator. Therefore, you typically enable the condition bit window for one instrument at a time to make sure that you know which instrument set its condition bit.

For more information, see:

- [About PatternTool](#)
- 
- [AWG6G Pattern Microcodes](#)
- 
- [ConditionBitEnabled](#)

<

>

# awg6G\_prog.3.08

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Programming Concepts : AWG6G Alarms and Error Reporting

### AWG6G Alarms and Error Reporting

An instrument alarm indicates a condition that can invalidate test results or cause physical damage to the instrument. The AWG6G does not report alarms. Instead, the AWG6G returns hardware errors to alert you to a condition you cannot ignore. The errors indicate that your test program has attempted to do something that the hardware cannot do.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.09

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

## AWG6G Programming : Programming Concepts : AWG6G Connections

### AWG6G Connections

This section includes the following topics:

- |   |                                                           |
|---|-----------------------------------------------------------|
| • | <a href="#">Single-Ended and Differential Connections</a> |
| • | <a href="#">DIB Access Connections</a>                    |

#### Single-Ended and Differential Connections

The AWG6G is normally a differential instrument, fanning out a signal through up to four differential pairs. Because AWG6G language elements apply to both channels in the differential pair, you do not need to create pin groups containing a positive and a negative channel from the same differential pair.

Each of the eight AWG6G channels includes a PPMU. If you want to use the PPMUs behind each channel independently, you must declare both channels. For more information, see [Programming the Channel Map](#).

If your device requires a single-ended input, you can use each of the four AWG6G outputs as a single-ended channel. To source a single-ended waveform, tie the inactive channel to ground on the DIB through 50ohms as close as possible to the AWG6G output and set the `ConnectionType` property to `tlAWGConnectionTypeSingleEnded`. IG-XL uses this information to produce the common mode voltage and amplitude you specify for your signal.

#### DIB Access Connections

The AWG6G does not use DUT relays and cannot be the primary instrument in a DIB Access line. However, the AWG6G can be the secondary instrument behind a DIB Access line. You must use close the DIB Access relays independently of the AWG6G (using utility bits on the DIB or an HSD

channel, for example) to connect the AWG6G output to the primary DIB Access instrument. After you connect the line, you can program the channel as you would a dedicated AWG6G channel. For more information, see:

- [Multiple Resources and DIB Access](#) in DataTool help
- [DIB Access Traces](#) in the DIB Design Guidelines

<

>

2015 Teradyne, Inc. All rights reserved.



# awg6G\_prog.3.10

<

>

## AWG6G Programming : Programming Concepts : AWG6G Reset Values

### AWG6G Reset Values

The following table lists the AWG6G reset values.

| AWG6G Reset Values  |                   |
|---------------------|-------------------|
| Parameter           | Reset Value       |
| LoadImpedance.Real  | 100ohms           |
| ConnectionType      | tlAWGDifferential |
| CommonMode          | 0V                |
| Amplitude           | ~0.399Vpk         |
| LevelFrequency      | 10MHz             |
| ConditionBitEnabled | Enabled           |
| Filter              | 1350MHz           |
| SampleRate          | 4.8GHz            |
| OutputDelay         | 0s                |

Because the AWG6G fans out a signal to four channels, resetting the instrument affects the output of all four channels.

Resetting the AWG6G does not affect settings stored in PSets or signals.

<

>

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.11



## [AWG6G Programming](#) : Programming Overview

### Programming Overview

To develop a test program that uses the AWG6G instrument, follow this basic procedure:

1. [Create a pin map to describe tester-to-device pin connections.](#)

[See Programming the Pin Map.](#)

2. [Create a channel map to describe channel connections.](#)

[See Programming the Channel Map.](#)

3. [Determine the parameters of the measurement.](#)

[See Creating a Wave Definition and Mixed Signal Timing Set.](#)

4. [Include AWG6G microcodes in your digital pattern. \(This step is optional.\)](#)

[See AWG6G Pattern Control.](#)

5. [Create a test procedure to perform your tests.](#)

6. [Add your tests to the Flow Table.](#)

7. [Validate and run your test program.](#)

[VBT syntax is described in the AWG](#) section of the VBT Syntax Reference.

|                 |                                          |  |
|-----------------|------------------------------------------|--|
| <div>&lt;</div> | <div>&gt;</div>                          |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |  |

# awg6G\_prog.3.12

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : AWG6G DataTool Programming

### AWG6G DataTool Programming

This section describes how to program several DataTool worksheets to handle basic AWG6G parameters. It includes the following topics:

- [Programming the Pin Map](#)
- [Programming the Channel Map](#)

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.13

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Programming](#) : [AWG6G DataTool Programming](#) : Programming the Pin Map

## Programming the Pin Map

A pin map describes the device pins by assigning a pin type to each device pin name.

Select the Analog pin type for each device pin name connected to an AWG6G instrument. If the AWG6G is the only resource behind this pin, select Analog as the pin type. If there are multiple resources behind this pin, select the most appropriate pin type based on the nature of the other resources.

Programming settings for either the Pos or Neg pin of a differential pin pair programs both pins of the pair. You do not need to program the pins individually or to create a differential pin group. However, programming both pins does not cause problems.

For more information, see [Pin Map Sheet](#) in the DataTool section.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.14

<

>

## AWG6G Programming : AWG6G DataTool Programming : Programming the Channel Map

### Programming the Channel Map

A Channel Map sheet assigns individual device pin names to test system channels and power supplies. For the AWG6G instrument, you must select one of two channel types from the Type column menu for each pin or pin group:

|        |                    |
|--------|--------------------|
| AWGNeg | AWG6G Negative Pin |
| AWGPos | AWG6G Positive Pin |

Because the AWG6G is a differential instrument, the “positive” input carries the signal and the “negative” input carries the inverse of the signal.

You can program a particular AWG6G instrument by specifying either the positive or negative pin or channel on that instrument. All AWG6G language elements apply to both channels, so you do not need to create differential pin groups containing a positive and a negative channel from the same differential pair. If you do program both the positive and negative pins of a differential pair, there are no negative effects.

To use the PPMUs behind both positive and negative channels, you must explicitly define both AWGPos and AWGNeg pins of a differential channel in the channel map. During validation, IG-XL verifies that each pin definition is correct (that is, the positive pin at SlotX.ChannelY is of type AWGPos and the negative pin at SlotX.ChannelZ is of type AWGNeg.)

For more information, see [Channel Map Sheet](#) in the DataTool section.

### Sharing AWG6G Resources

You can share a single AWG6G instrument among multiple sites or pins on the same site using standard IG-XL and UltraFLEX procedures. If you use a relay to modify the connection between a single AWG6G resource and multiple sites or pins on a DIB, you must use utility bits or other

standard DIB control mechanisms to control the relay. No AWG6G language is available to change the state of a relay.

For more information, see [Resource Sharing by Pins and Sites](#) in DataTool help.

Using the AWG6G with DIB Access

You can use the AWG6G as a secondary instrument in a DIB access connection. As the AWG6G instrument has no DIB access path, you cannot use the AWG6G as the primary instrument in a DIB access connection. Declaring an AWG6G channel as a primary DIB access instrument in the Channel Map returns a validation error.

For more information, see [DIB Access](#) in DataTool help.

---

|                                                                                   |  |                                                                                   |  |
|-----------------------------------------------------------------------------------|--|-----------------------------------------------------------------------------------|--|
|  |  |  |  |
|                                                                                   |  | 2015 Teradyne, Inc. All rights reserved.                                          |  |



# awg6G\_prog.3.15

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Creating a Wave Definition and Mixed Signal Timing Set

### Creating a Wave Definition and Mixed Signal Timing Set

An AWG6G generates and delivers analog signals of a specific shape, frequency, and amplitude to the DUT. These signals start as wave definitions, which define the basic shape of a waveform. You add a timing solution to the waveform to create a timing set for the test. The AWG6G uses the timing set to create a reusable signal to which you can apply various amplitudes.

The following topics are available:

- |   |                                                                                   |
|---|-----------------------------------------------------------------------------------|
| • | <a href="#">Setting Up a Wave Definitions Sheet</a>                               |
| • | <a href="#">Creating a Timing Set Using the Mixed Signal Workshop</a>             |
| • | <a href="#">Associating a Wave Definition with a Mixed Signal Timing Solution</a> |

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

# awg6G\_prog.3.16

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Creating a Wave Definition and Mixed Signal Timing Set : Setting Up a Wave Definitions Sheet

### Setting Up a Wave Definitions Sheet

A Wave Definitions sheet contains the wave shape information for waveform definitions that were created using the WaveDesigner. A Wave Definitions sheet stores wave definition information, and the test program accesses the information when the test is run.

To work with wave definitions, open a Wave Definitions sheet or insert a new one. To insert a new Wave Definitions sheet:

1. [Select](#) Insert>Insert IG-XL Worksheet in the Sheet group on the IG-XL Main tab.
2. [Select](#) Wave Definition from the Sheet type menu in the Insert IG-XL Worksheet dialog box.
3. [Type a name for the sheet in the](#) Sheet name field.

[If you leave](#) Sheet name blank, IG-XL uses the name WaveDef.

4. [Click](#) OK.

IG-XL creates the sheet and includes a wave definition called aSine.

For more information, see:

- [About WaveDesigner](#)
- [Waveform Definitions](#)

|   |                                        |
|---|----------------------------------------|
| • | <a href="#">Wave Definitions Sheet</a> |
|---|----------------------------------------|

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_prog.3.17

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Creating a Wave Definition and Mixed Signal Timing Set : Creating a Timing Set Using the Mixed Signal Workshop

### Creating a Timing Set Using the Mixed Signal Workshop

The Mixed Signal Workshop (MSW) tool helps you to build reusable mixed signal timing solutions that meet coherence requirements.

The WaveDesigner tool creates wave definitions that are independent of timing and instrument specific information. The MSW uses these wave definitions and applies the necessary timing information to them to generate samples.

The output of the MSW is a mixed signal timing set that contains the signal’s structural characteristics and the details of the clocking solution that you created. Timing sets are stored on a Mixed Signal Timing sheet in the IG-XL workbook and become a reusable timing solution for future tests.

To create the required timing set:

1. Insert and set up a Mixed Signal Timing sheet in the IG-XL workbook:
- a. Select Insert>Insert IG-XL Worksheet in the Sheet group on the IG-XL Main tab.

b. Select Mixed Signal Timing from the Sheet type menu in the Insert IG-XL Worksheet dialog box.

c. Type a name for the sheet in the Sheet name box (optional). Default name is Mixed Signal Timing.

d. Click OK.

- e. Type a timing set name in the Set Name column and press Enter.
- f. Right-click the timing set name and select Edit from the context menu to start the Mixed Signal Workshop (MSW) tool in TDE.

The MSW display appears and displays the DUT and capture resources blocks.

2. Add an AWG6G resource to the MSW display:

- a. Select MSW>Add Source.
- b. Select AWG from the Select Resource dialog box.

Leave the Resource ID box empty unless this timing set uses additional AWG6G instruments. If the timing set does use additional instruments, assign the current source instrument a unique ID number to distinguish it from the others.

- c. Click OK in the Select Resource dialog box.

The AWG6G resource block appears on the MSW display.

3. Select the appropriate wave definition from the Wave Definition menu on the AWG resource block.
4. Specify the Source timing.

- a. Click the Solve button in the Source resource block to start the Solver tool.
- b. Select the Fs radio button to solve for the sample rate (Fs).
- c. In the Repeat Frequency frame's Target field, type the appropriate value for the desired repeat frequency (FR) to choose from a list of desired capture frequencies. Choosing from the list of desired capture frequencies often provides the most efficient solution because it allows IG-XL to program the instrument according to the capture's sampling parameter constraints. Note that, for a sine wave, the repeat frequency is the same as the fundamental frequency (Fi).

- d.

Constrain the sample count to return a useful number of solutions by modifying the starting and stopping values in the Sample Count frame. If the MSW default search parameters for sample size do not yield a solution, increase the upper sample size limit.
- e.

Enter the number of cycles (M) for the source signal. This is usually 1, which produces the minimum number of source samples and allows the source instrument to repeat the segment indefinitely. The MSW Solver's tool default is 1.
- f.

[Click Show Choices](#), select a solution from the table, then click OK.

The most common solutions are those with a minimum N and/or minimum FR error. The best solution, however, can depend on the specifics of a test's needs. If the Solver finds no solutions, change the values you give the Solver. Changing the sample size range frequently provides usable results. You can also check [Show All](#) to see why some solutions were invalid.

5.

[Save the timing set to the Mixed Signal Timing sheet by clicking the Save to Excel button](#) in the toolbar. Note that this does not save the workbook to disk; to keep your changes, you must also save the workbook.

For more information, see [Mixed Signal Workshop](#).

<

>

2015 Teradyne, Inc. All rights reserved.

## awg6G\_prog.3.18



### AWG6G Programming : Creating a Wave Definition and Mixed Signal Timing Set : Associating a Wave Definition with a Mixed Signal Timing Solution

#### Associating a Wave Definition with a Mixed Signal Timing Solution

If you use the Mixed Signal Workshop to create a timing solution for an AWG6G, you must associate a Wave Definition with the timing solution.

When you create a waveform, IG-XL makes the waveform available to all instruments. To source a signal using a wave definition from this pool, assign a wave definition name to a signal and specify the number of samples and the signal's frequency and amplitude.

You can assign a wave definition to an AWG signal in the following ways.

- Use the `ApplyMixedSignalTiming` method

Set the `ApplyMixedSignalTiming` property:

```
TheHdw.AWG.Pins("PinSpec").Signals("SignalName").ApplyMixedSignalTiming
```

This property applies the AWG portion of the timing solution defined in the current mixed signal timing solution to the specified signal. An AWG mixed signal timing solution includes the wave definition name and other information about the signal.

- Use the `WaveDefinition` property

Set the following properties for the Signal definition:

```
...Signals("mySignal").WaveDefiniton = "WaveDefName"
```

```
...Signals("mySignal").SampleRate = x
```

```
...Signals("mySignal").NumSamples = y
```

Both methods assign a wave definition name, not the wave definition itself. IG-XL uses this name to retrieve the actual wave definition when your test program loads the waveform. Therefore,

assigning an incorrect wave definition name does not cause an error at this point in the test program.

Loading Waveforms

When IG-XL loads waveforms, it verifies that the hardware can support the number of samples in the waveforms. For more information, see [AWG6G Specifications](#).

IG-XL loads waveforms when your test program does any of the following:

- Calls the VBT method `Signals.Start`:

`TheHdw.AWG.Pins("myPin").Signals("mySignal").Start`

If there is not enough room in waveform memory, IG-XL returns an error.

When IG-XL must load samples before a start, there is a delay between the call to the `Start` method and when hardware starts sourcing the signal.

- Starts a pattern that refers to a signal

Before a pattern starts, IG-XL loads that any signals that the pattern calls.

If the memory does not have enough room to load all the waveforms that the pattern calls, IG-XL returns an error.

- Calls the VBT method `Signals.LoadSamples`

For example:

`TheHdw.AWG.Pins("myPin").Signals("mySignal").LoadSamples`

IG-XL loads samples only if the samples are not already loaded. If the samples already exist in memory, IG-XL does not reload them.



# awg6G\_prog.3.19

|                      |                      |
|----------------------|----------------------|
| <a href="#">&lt;</a> | <a href="#">&gt;</a> |
|----------------------|----------------------|

## AWG6G Programming : AWG6G Pattern Control

### AWG6G Pattern Control

You can control an AWG6G using pattern microcodes, allowing changes to instrument states and parameters during a program. You can obtain repeatable timing by programming an instrument using pattern microcodes and parameter sets (PSets).

You create a pattern independently of the test program using the PatternTool or a text editor. Each digital vector in a pattern defines the data for each channel listed in the pattern, and can also define a pattern microcode that controls a separate instrument during pattern execution. To use pattern microcodes to control the AWG6G on UltraFLEX, you must include a digital pin in the pin map, the channel map, and the pin list of the pattern module.

The following topics are available:

- [AWG6G Pattern Microcodes](#)
- [AWG6G ASCII Pattern Syntax](#)
- [PSets](#)

For more information, see:

- [Pattern Compiler](#)
- [Pattern Reverse Compiler](#)

|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_prog.3.20

<

>

## AWG6G Programming : AWG6G Pattern Control : AWG6G Pattern Microcodes

### AWG6G Pattern Microcodes

A microcode controls a single function, such a starting or stopping the sourcing of a signal. series of microcode commands are available for use inside a pattern. Patterns allow you to precisely control the timing of when a signal is sourced. Use patterns to establish repeatable timing or to synchronize with other instruments (such as the digital subsystem) in the tester.

The following table lists the extended microcodes used to issue commands to an AWG6G instrument from a pattern.

| AWG6G Pattern Microcodes |                                                                                                                                                                                                            |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microcode                | Functional Description                                                                                                                                                                                     |
| Disable_Inst_Cond        | Disables the AWG6G from asserting the instrument condition. See Condition Bit Programming.                                                                                                                 |
| Enable_Inst_Cond         | Enables the instrument condition. Use the ConditionData VBT property to determine whether the AWG6G asserts the instrument condition at the beginning or end of a waveform. See Condition Bit Programming. |
| PSet <PSetName>          | Executes the parameter set (PSet) which changes the instrument’s setup (voltage range, amplitude, and sample rate). This microcode requires a single argument, which is the name of the PSet to apply.     |
| Resync                   | Forces phase alignment of the AWG6G clock with the HSD T0 clock. See Microcode Restrictions for limitations.                                                                                               |

|                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Start <SignalName>   | <p>The instrument immediately (on the next available logical analog clock) starts sourcing the named signal in SMEM and repeats it continuously until a Stop microcode is executed, the instrument is reset, or another signal is started.</p> <p>This microcode takes an optional argument, which is the name of the signal to be sourced. If the signal name is unspecified, the default signal is used. See TheHdw.AWG.Pins.Signals.DefaultSignal.</p> |
| Start1 <SignalName>  | <p>The instrument immediately (on the next available logical analog clock) starts sourcing the named signal in SMEM and sources it exactly once.</p> <p>This microcode takes an optional argument, which is the name of the signal to source. If the signal name is unspecified, the default signal is used. See TheHdw.AWG.Pins.Signals.DefaultSignal.</p>                                                                                               |
| StartE <SignalName>  | <p>At the end of the currently sourcing signal's segment, the instrument sources the named signal in SMEM and repeats it continuously until a Stop microcode is executed, the instrument is reset, or another signal is started.</p> <p>This microcode takes an optional argument, which is the name of the signal to be sourced. If the signal name is unspecified, the default signal is used. See TheHdw.AWG.Pins.Signals.DefaultSignal.</p>           |
| StartE1 <SignalName> | <p>At the end of the currently sourcing signal's segment, the instrument sources the named signal in SMEM and sources it exactly once.</p> <p>This microcode takes an optional argument, which is the name of the signal to source. If the signal name is unspecified, the default signal is used. See TheHdw.AWG.Pins.Signals.DefaultSignal.</p>                                                                                                         |
| Stop                 | <p>The instrument stops sourcing the currently sourcing signal immediately. Stop does not affect a signal that is running once.</p>                                                                                                                                                                                                                                                                                                                       |
| StopE                | <p>The instrument stops sourcing the currently sourced signal at the end of the signal.</p>                                                                                                                                                                                                                                                                                                                                                               |

A parameter set (PSet) is a unique microcode that contains a set of values that describe a specific instrument setup. PSets allow a test program to change instrument setups within a pattern

execution. Although a PSet is a hardware capability that changes the setup of an instrument during a pattern burst, you define it using VBT language and invoke it using PSet microcode.

### Synchronizing the Analog Clock

The Resync microcode provides the ability to synchronize the tester's analog and digital clocks. It ensures that the first tick of all synchronized boards appears at a repeatable multiple of the digital subsystem clock. This allows you to create repeatable and portable timing solutions.

Synchronizing clocks does not affect the settings of any particular board with its own clock, including the AWG6G. Settings such as the sampling rate remain as you programmed them. The Resync microcode affects the timing source for the entire board, which changes the timing on all channels.

For more information, see [Microcode Restrictions](#).

### Microcode Restrictions

The Resync microcode aligns the AWG6G analog clock with the digital subsystem clock to help ensure proper timing and repeatability. The UltraFLEX test system imposes the following restrictions on use of the Resync microcode:

- [Use the same](#) Resync arrangement in all patterns that you load at the same time. An "arrangement" is one or more instruments that use the Resync microcode. If all instruments use Resync, then you must resynchronize all instruments each time you resynchronize any of them. You can call a Resync arrangement more than once in a pattern, and the arrangement can exist in any, all, or none of the other patterns you load together.

[An arrangement must be the same in all patterns you intend to load together.](#) If you start a pattern that includes a different Resync arrangement, IG-XL returns a run-time error.

[Before you load patterns that perform](#) Resync independently, you must unload all patterns that synchronize together.

[For example, you load](#) Pat1, Pat2, and Pat3 together. Pat1 has Resync on pin srcpos1 in vectors 1 and 5. Pat2 has no Resync. Pat3 has Resync on pin srcpos1 in vector 3. This works because the Resync microcode appears on the same pins across all loaded patterns. If you move the Resync to pin srcpos2 in Pat3, the pattern fails.

- [Do not place a](#) Resync on the same vector as a Halt.

- [Do not mix the](#) PSet, Enable\_Inst\_Cond, Disable\_Inst\_Cond, and Resync microcodes across the channels in the same vector. These microcodes can exist on the same vector as a NOP. For example, do not put Enable\_Inst\_Cond on channel 1 and PSet on channel 2 in the same pattern vector.

- Do not mix PSet names within a vector on channels on the same board. For example, if channel 1 calls PSet PS1, channel 3 cannot call PSet PS2 on the same vector.

You can mix Enable\_Alarm, Disable\_Alarm, and Trig microcodes on the channels of the same board in the same pattern vector. Trig labels can differ across channels also.

---

|                                                                                   |                                                                                   |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--|
|  |  |  |
|                                                                                   | 2015 Teradyne, Inc. All rights reserved.                                          |  |

# awg6G\_prog.3.21

|                                                                                   |                                                                                   |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|  |  |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|

## AWG6G Programming : AWG6G Pattern Control : AWG6G ASCII Pattern Syntax

### AWG6G ASCII Pattern Syntax

AWG6G pattern microcodes are instructions you can place on an individual vector of a pattern. For that vector, a microcode instructs the AWG6G to perform a particular action, such as starting a new signal.

You can use AWG6G microcodes only with pins that the instruments statement has specified as AWG. For example:

```
instruments = {
    DUT_AIN:AWG;
}
```

For more information, see [Instruments Statement](#) in the Pattern Language section.

If a vector contains any AWG6G related microcode other than NOP, that vector is placed in SRM.

The format for specifying a AWG6G microcode on a vector is:

(pin-list : AWG = microcode)

The following code demonstrates the ASCII pattern microcode syntax for the AWG6G.

```
import tset TSet_1;
instruments = { AWG_Pos_Pins:AWG 1; }
srm_vector
PRML_Into_Scope_Pat_Ctrl_module ( $tset, Trig)
{
    > TSet_1 0 ;
start_label start_awg_prml_scope_view:
(AWG_Pos_Pins = Resync)
clr_cond_flags (cpuA_cond) > TSet_1 1 ;
(AWG_Pos_Pins = Start AWG_PRML)
branch_expr = (!cpuA_cond) > TSet_1 0 ;
```

```
repeat 25000 > TSet_1 1 ;
global loop_prml:
> TSet_1 1 ;
repeat 64000 > TSet_1 0 ;
if (branch_expr) jump loop_prml > TSet_1 1 ;
(AWG_Pos_Pins = Stop)
> TSet_1 1 ;
halt > TSet_1 0 ;
> TSet_1 X ;
.
.
> TSet_1 X ;
}
```



# awg6G\_prog.3.22

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : AWG6G Pattern Control : PSets

### PSets

You can use parameter sets (PSets) to control the AWG6G from a pattern. PSets allow you to configure the AWG6G from within a running pattern, providing repeatable timing.

### What is a PSet?

A PSet is a named set of property values that you program using VBT and implement in hardware using either a VBT statement or a pattern microcode. This allows you to define a set of hardware settings once and apply the settings repeatedly.

Each AWG6G PSet contains a value for each of the following parameters:

- |   |                     |
|---|---------------------|
| • | Amplitude           |
| • | Filter              |
| • | Sample rate         |
| • | Common mode voltage |
| • | Level frequency     |
| • | Output delay        |

IG-XL uses default values for any property you do not explicitly program. For more information, see [AWG6G Reset Values](#).

When you call a PSet from a pattern, you must always name the PSet you want to call; you cannot specify a default PSet.

Each AWG6G has enough PSet memory to store 292 unique PSets for an entire test program. Other parts of the test system infrastructure limit the total number of unique usages of PSets and microcodes used across all loaded patterns for all instruments in a single timing realm to 248.

For more information, see [Parameter Sets \(PSets\)](#).

PSet Error Checking

Because you define PSets dynamically during a test program run, validation does not include error checking or processing. This happens when you start a pattern that contains a PSet. At that time, IG-XL verifies that you have defined all signal and PSet names that appear in the pattern. If you leave any PSets or signals undefined, IG-XL returns a run-time error that notes the name of the pattern that contains the undefined item and the name of the procedure or the VBT function from which you call the pattern.

Do not configure PSets during a pattern burst, as this can lead to a conflict between the Visual Basic programming and implementing the PSet in hardware. If this happens, IG-XL returns a run-time error.

PSet Programming

To create a PSet and assign values to PSet properties, use the PSets VBT property. For more information, see [TheHdw.AWG.Pins.PSets](#) in the VBT Syntax Reference.

To program hardware with the values in a PSet, use:

- [The PSet microcode in an HSD pattern \(see \[AWG6G Pattern Microcodes\]\(#\)\)](#)
- [The VBT method `TheHdw.AWG.Pins.PSets.Item.Apply`](#)

# awg6G\_prog.3.23

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Programming : Using AWG6G in Concurrent Testing

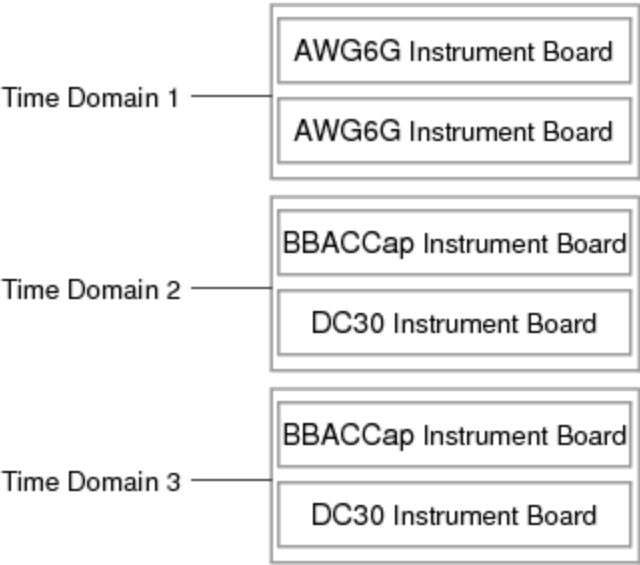
### Using AWG6G in Concurrent Testing

To support concurrent testing, you can assign AWG6G channels to a time domain. This means that AWG6G instruments can be part of a concurrent pattern burst. However, only one pattern in the concurrent burst can contain AWG6G instruments in it at one time. This limitation means that AWG6G channels cannot be in more that one time domain for each concurrent burst.

Concurrent features for AWG6G include:

- An AWG6G board can be configured for one time domain
- Multiple AWG6G instruments can be grouped into a common time domain
- AWG6G is restricted to using the Global flow domain and synchronous pattern starts

The following figure shows a configuration with two AWG6G instruments assigned to its one time domain and other UltraFLEX instruments assigned to other time domains.



AWG6G Concurrent Testing Configuration Example  
For more information, see [About Concurrent Test](#).

# awg6G\_theory.2.01

|                      |                      |
|----------------------|----------------------|
| <a href="#">&lt;</a> | <a href="#">&gt;</a> |
|----------------------|----------------------|

## AWG6G Theory of Operation

### AWG6G Theory of Operation

This section describes the operation of the AWG6G in the following sections:

- |   |                                         |
|---|-----------------------------------------|
| • | <a href="#">Hardware Block Diagrams</a> |
| • | <a href="#">Functional Description</a>  |
| • | <a href="#">Operating Concepts</a>      |

|                      |                      |
|----------------------|----------------------|
| <a href="#">&lt;</a> | <a href="#">&gt;</a> |
|----------------------|----------------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.02

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Hardware Block Diagrams

### Hardware Block Diagrams

The AWG6G is a fan-out instrument with one instrument per board and a four channel fan-out. The only channel-independent controls are the connections. If you use more than one differential channel of an AWG instrument, it is good practice to specify at least one pin from each differential channel when programming the instrument (using either a comma-separated list or a pin group on the pin map).

This section includes the following diagrams:

- |   |                                           |
|---|-------------------------------------------|
| • | <a href="#">AWG6G Instrument Diagrams</a> |
| • | <a href="#">AWG6G Analog Path Diagram</a> |
| • | <a href="#">Test Head Configurations</a>  |

For a description of AWG6G PPMUs, see [AWG6G PPMU](#).

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

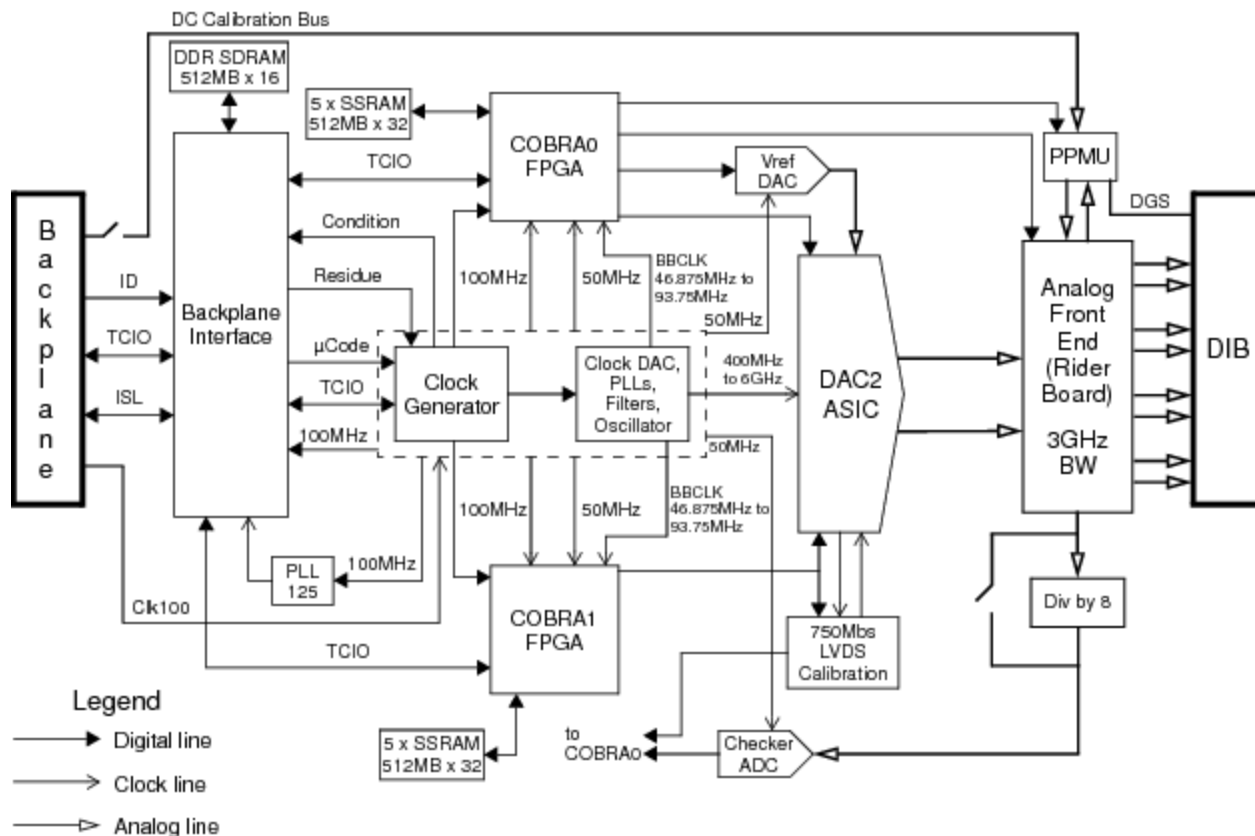
|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.03

[AWG6G Theory of Operation](#) : [Hardware Block Diagrams](#) : [AWG6G Instrument Diagrams](#)

## AWG6G Instrument Diagrams

The following block diagram describes the overall AWG6G instrument. For AWG6G Plus frequencies, see the notes following the diagram.



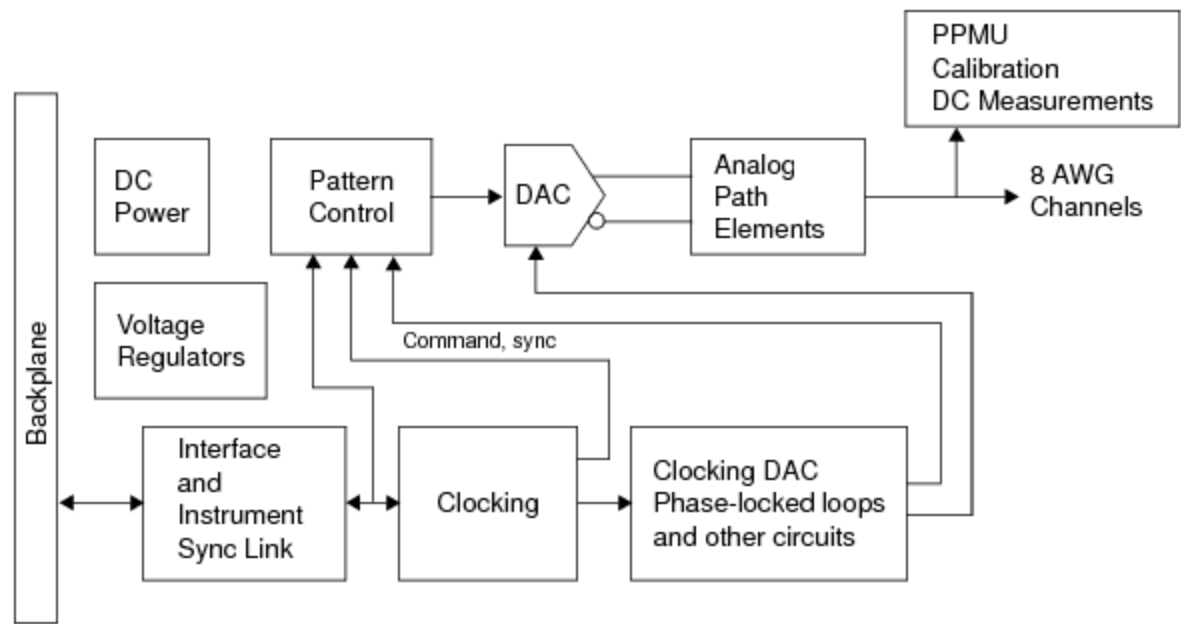
## AWG6G Instrument Block Diagram

The AWG6G Plus uses the same frequencies, except:

- LVDS Calibration frequency is 825MHz

- BBCLK frequency range is 46.875MHz to 103.125MHz
- DAC2 input frequency range is 400MHz to 6.6GHz

The following diagram provides a simplified functional view of the instrument.



AWG6G Simplified Block Diagram



[AWG6G Theory of Operation](#) : [Hardware Block Diagrams](#) : [AWG6G Analog Path Diagram](#)

The following diagram describes the analog path in more detail.



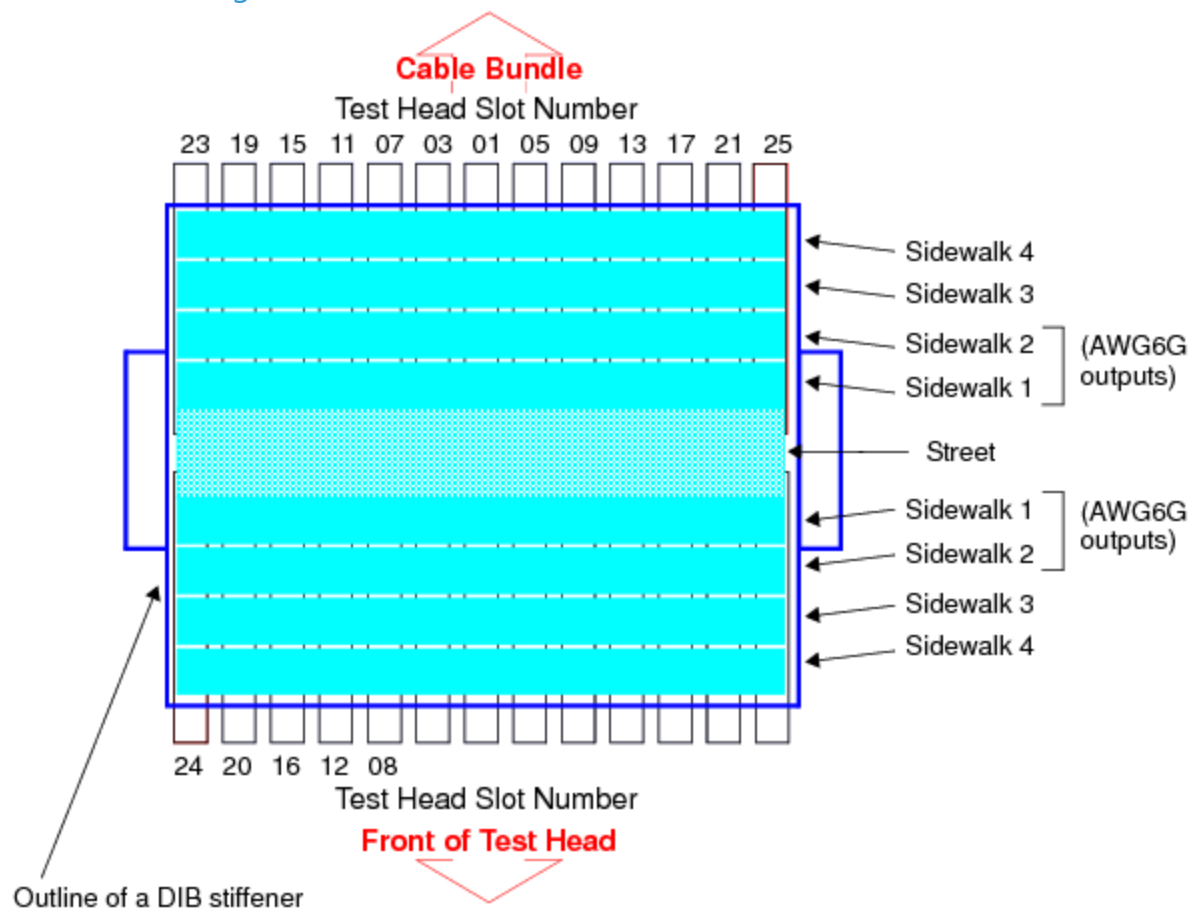
|                 |                                          |                 |  |
|-----------------|------------------------------------------|-----------------|--|
| <div>&lt;</div> |                                          | <div>&gt;</div> |  |
|                 | 2015 Teradyne, Inc. All rights reserved. |                 |  |

# awg6G\_theory.2.05



AWG6G Theory of Operation : [Hardware Block Diagrams](#) : Test Head Configurations

## Test Head Configurations



Street Locations for AWG 6G Outputs (not to scale)

Sidewalk 2 connections are similar to the sidewalk 1 connections and use the same SMP type connectors. Sidewalk 2 connections have a lower frequency response than sidewalk 1 connections.



|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.06

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Functional Description

### Functional Description

The AWG6G consists of a test head channel card with circuitry for:

- |   |                                              |
|---|----------------------------------------------|
| • | Digital processing                           |
| • | Waveform memory                              |
| • | Data multiplexer                             |
| • | Digital-to-analog converter (DAC)            |
| • | Phase-locked loop (PLL) and clock generation |
| • | Host computer communications (TCIO)          |
| • | Instrument Sync-Link (ISL)                   |
| • | Low-pass filters (LPF)                       |
| • | Attenuators                                  |
| • | Amplifiers                                   |

- [Calibration circuitry](#)

This section describes the following topics:

- [PLL and Clock Generation](#)
- [Waveform Generation Control](#)
- [Waveform Memory Data and Multiplexing](#)
- [Output Waveform DAC and Signal Paths](#)
- [AWG6G PPMU](#)
- [Test System Interface](#)
- [Power Board](#)

<

>

# awg6G\_theory.2.07

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Functional Description : PLL and Clock Generation

### PLL and Clock Generation

The AWG6G generates the A(n) clock on the motherboard. A copy of the 100MHz test system reference clock is passed into a DDS baseband generation circuit to generate the baseband A(n) clock, which always runs between 11.71875MHz and 23.4375MHz regardless of the sample clock rate (Fs) setting. The A(n) rate determines the timing granularity of microcode tossing in the pattern. Fs is always a power-of-2 multiple of A(n). To determine the A(n) rate from a given Fs rate, find a power-of-two number, M, from 32 to 256 such that Fs/M falls in the A(n) octave range. Fs/M is the A(n) rate.

The baseband A(n) clock is multiplied by a PLL to generate the Fs (400MHz to 6000MHz for the AWG6G, 400MHz to 6600MHz for the AWG6G Plus). The outputs of the PLL are fed to the main DAC and the calibration circuitry.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.08

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Theory of Operation : Functional Description : Waveform Generation Control](#)

Waveform Generation Control

The sequencer controls real time execution of AWG6G waveforms. You define a waveform and the microcoded program that controls it. During run time, this program is executed in the sequencer. The microcoded program includes control functions, such as looping waveforms and branching between waveforms segments. The AWG6G supports simple segments. A simple segment is a data set, stored in waveform memory that defines an analog waveform having evenly-spaced time increments. When sourced, the data set is sent to the main DAC and converted into corresponding analog voltages. A simple segment can be executed once or looped continuously until it is stopped or another waveform is started. See [Wave Segments](#).

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|



# awg6G\_theory.2.09

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Theory of Operation : Functional Description](#) : Waveform Memory Data and Multiplexing

Waveform Memory Data and Multiplexing

On the AWG6G, a waveform you program is stored in a 16 Megasample memory on the card. This memory provides 16,777,216 storage locations. The first 16,711,680 are available to you for storing waveform data while the remaining 65,536 are reserved for calibration data.

The 64:1 data multiplexer distributes waveform memory data samples to the main DAC. The input data are 640 bits wide at a maximum frequency of 100MHz, while the output data are 80 bits wide at a maximum frequency of 750MHz for the AWG6G, 825MHz for the AWG6G Plus.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.10

|   |   |
|---|---|
| < | > |
|---|---|

## AWG6G Theory of Operation : Functional Description : Output Waveform DAC and Signal Paths

### Output Waveform DAC and Signal Paths

Waveform samples are sourced out of the waveform memory and applied to the DAC, which converts the digital sample stream to an analog waveform.

At this point, the signal can either be fed into a programmable filter bank (either 1350MHz or 2000MHz) or bypass the filters entirely. The signal is converted to differential, amplified, and fed to the output switching relays. Here the programmable DC baseline can be applied.

**✔ Note:**

The DC baseline must be at least 0V, and not more than 2V with a single-ended load of 50ohms or a differential load of 100ohms.

The leveling detector is used to set the output signal level to -2dBm across all frequencies. The temperature invariant absolute level reference oscillator (ALRO) is used in calibration.

|   |   |
|---|---|
| < | > |
|---|---|

# awg6G\_theory.2.11

&lt;

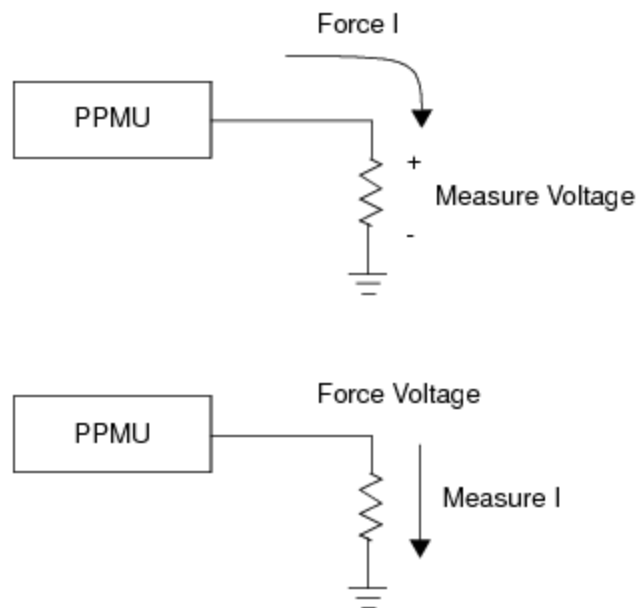
&gt;

## AWG6G Theory of Operation : Functional Description : AWG6G PPMU

### AWG6G PPMU

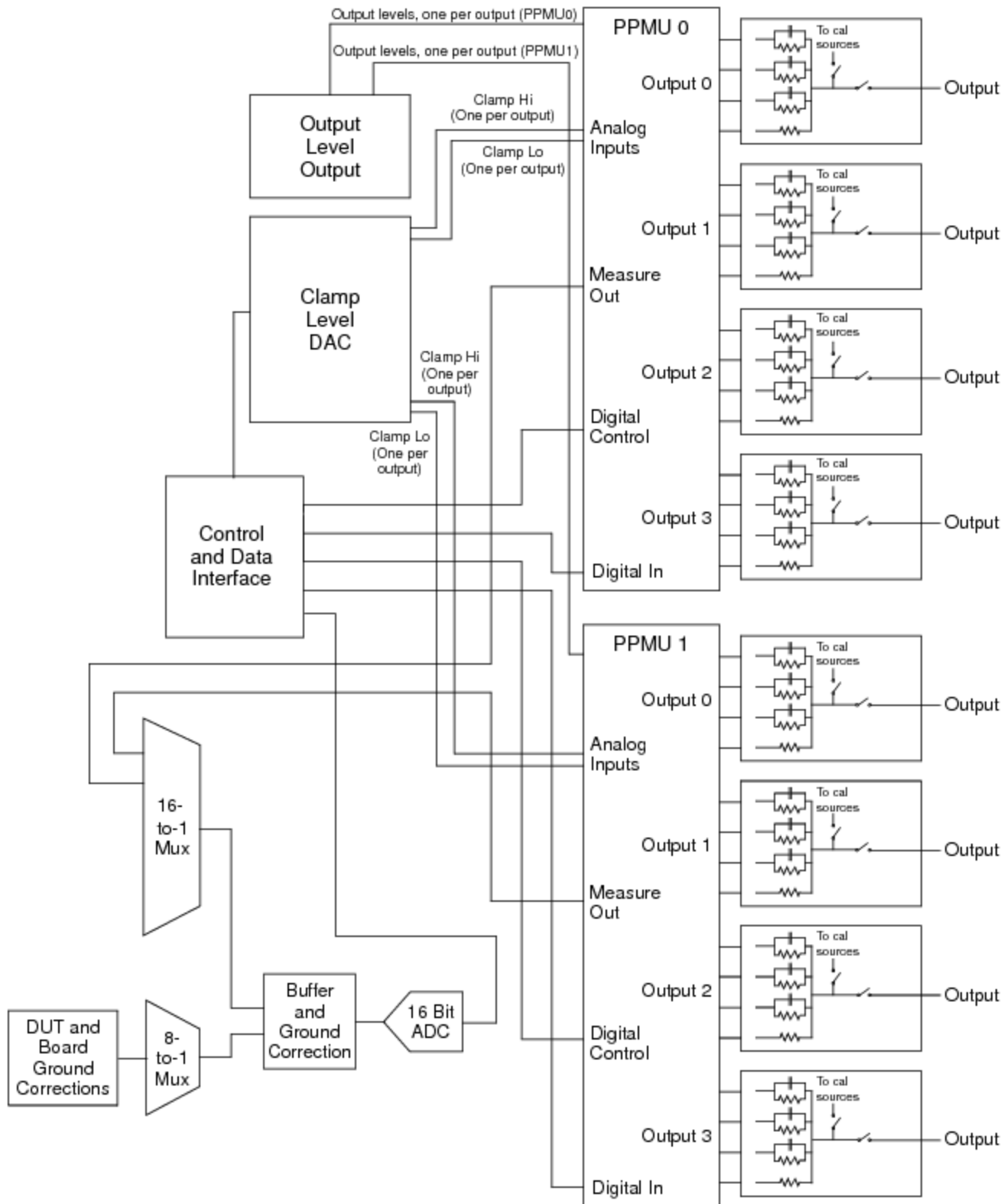
Every AWG6G output is connected to a four quadrant force-voltage and measure-current or force-current and measure-voltage pin parametric measurement circuit (PPMU). The PPMU can be used to:

- Force voltage while measuring current (for leakage measurements)
- Force current while measuring voltage (for open/shorts tests)



### PPMU Usage

The following diagram describes the arrangement of PPMUs on the AWG6G.



### AWG6G PPMU Block Diagram

For more information, see [About the PPMU](#).



|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.12

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Theory of Operation : Functional Description](#) : Test System Interface

Test System Interface

The AWG6G connects to the test system through several buses including:

- |   |                                                                                     |
|---|-------------------------------------------------------------------------------------|
| • | Instrument Sync Link: a test system bus that supplies microcode and synchronization |
| • | TCIO: the bus that carries instrument setup information                             |

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.13

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

[AWG6G Theory of Operation : Functional Description : Power Board](#)

Power Board

Each AWG6G has an integral power board that steps down the test system +48V supply to all the necessary voltage levels for the AWG6G circuitry.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.14

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Operating Concepts

### Operating Concepts

This section describes some general operating concepts as an aid to programming and using the AWG6G. These concepts are:

- [Wave Segments](#)
- [Setup Calibration](#)
- [Waveform Start Methods](#)

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|



# awg6G\_theory.2.15

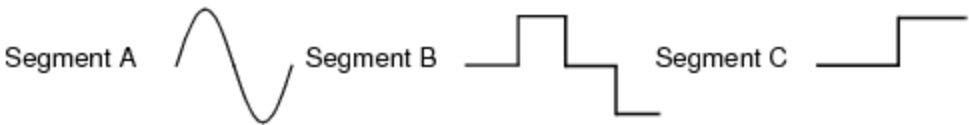
|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Operating Concepts : Wave Segments

### Wave Segments

Before you program an AWG, you usually know what shape the signal must have to provide the proper stimulus to the device. The wave must be stored in waveform memory as a wave segment (an array of data samples) before the AWG6G can source it.

The AWG6G’s sequencer enables the instrument to construct waveforms using simple wave segments, as in the following figure. This allows efficient use of waveform memory.



### Waveform Segment Example

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|

# awg6G\_theory.2.16

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Operating Concepts : Setup Calibration

### Setup Calibration

Setup, or levels, calibration is dependent on how the run-time parameters are programmed for each test. Each combination of run-time parameters results in a unique instrument setup, which may need a unique setup calibration based on the programmed settings. This type of calibration includes a DC setup and an AC setup.

For more information, see Automatic Calibration.

### DC Setup Calibration

DC setup calibration ensures that any DC offset added to the AWG6G signal is accurate within specifications. DC setup calibration can be run every time the instrument is set up or automatically whenever it is needed as determined by IG-XL. Automatic DC calibration may occur if a change is made in amplitude, filter setting, DC offset, DC calibration type, or leveling.

### Use of Tester Resources During DC Calibration

The AWG6G uses tester resources to perform DC setup calibration. These resources include the test system voltmeter and the DC crosspoint connections. IG-XL manages the use of these resources and, in most cases, this has no effect on the test program. However, AWG6G DC calibration may occasionally produce undesirable side effects.

IG-XL performs DC setup calibration by connecting each instrument channel to its crosspoint, connecting the crosspoint to the test system voltmeter, programming the meter, and performing the calibration to produce the desired offset. Before calibration, IG-XL saves the state of AWG6G connections, the crosspoint connections, and the meter. After calibration, IG-XL restores these states.

Calibration uses the following system resources:

- |                                                                    |
|--------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• AWG6G instrument</li></ul> |
|--------------------------------------------------------------------|

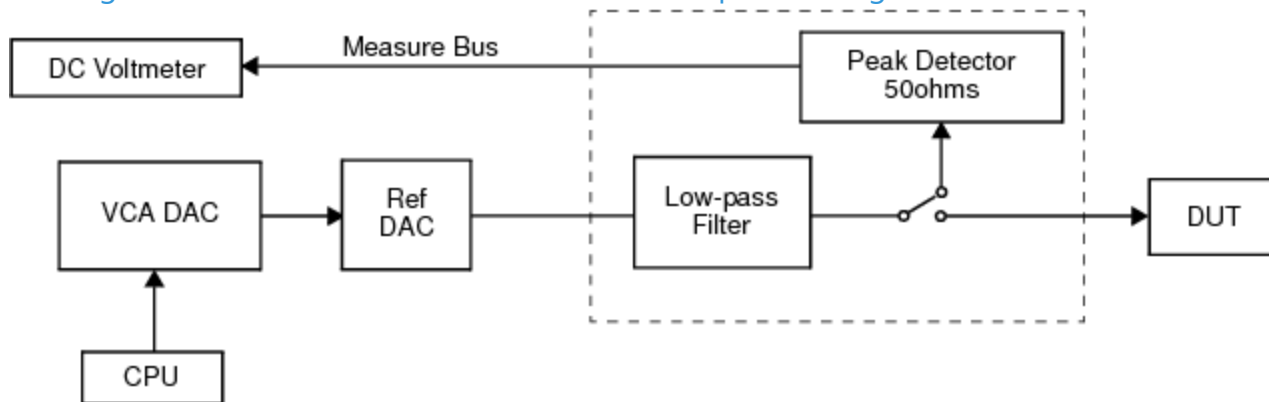
- One or both crosspoints allocated to the test head slot in which the AWG6G channel card resides
- Test system voltmeter

Any test program using these resources may see a temporary disconnect during DC calibration.

#### AC Setup Calibration (Leveling)

Leveling is a closed-loop internal run-time AC calibration that ensures the absolute amplitude accuracy of the AWG6G. The process removes a significant amount of the frequency response and path loss from the channel card to the output of the signal delivery frame. To program leveling, you must specify a frequency and a mode. If leveling is used, it can be programmed to occur every time the instrument is set up or automatically whenever it is needed as determined by IG-XL. Leveling may occur if a change is made in filter setting, leveling frequency, mode, sampling rate, AC clock frequency, or DC settings.

The leveling circuit is shown in the following figure. Leveling occurs through the internal 50ohm load on the channel card and is executed at a user-specified frequency. During leveling, the signal is disconnected temporarily from the DUT. A waveform is sourced from the channel card, and to the 50ohm terminated peak detector. The type of waveform sourced depends on the programmed leveling mode and is either a sine wave or a user-specified segment.



#### AWG6G Leveling Circuit

The peak output voltage of the detector is measured by the system dc voltmeter. Information stored on an instrument EEPROM during factory calibration is used to determine the detector voltage that corresponds to a -2dBm (0.5V peak into 50ohms) signal at the frequency of interest. A successive approximation search is used to adjust the gain of the instrument until the dc voltmeter measures this value. The instrument settings used to obtain this level are then internally saved as a "level table" for that particular waveform and the corresponding run-time parameters. After leveling is complete, the signal is disconnected from the peak detector and is reconnected to the DUT. The leveling table for a particular setup remains valid until a new test program is loaded, the temperature changes, or the AWG6G is calibrated manually or through the autocal utility.

For more information, see [Setting the Leveling Frequency](#).

Use of Tester Resources During Leveling

The AWG6G uses tester resources to perform ac setup calibrations. These resources include the test system voltmeter and the Measure Bus. IG-XL manages the use of these resources and, in most cases, this has no effect on the test program. However, AWG6G ac calibration may occasionally produce undesirable side effects.

During AC setup calibration, a signal from the AWG6G peak detector circuit is connected to the test system voltmeter through the measure bus. As with DC calibration, the state of these resources is automatically saved before calibration and restored afterward. However, any test program using these resources may see a temporary disconnect during leveling.

For most applications, use the automatic leveling option to enable automatic calibration. In this mode, the run-time calibration described above occurs only on the first test program execution or the first execution after autocalibration of the AWG6G. During subsequent executions, a look-up table provides the calibration constants and run-time calibration is not needed. The automatic option will not disturb devices sensitive to the disruptions to test system resources during AWG6G calibration.

In a few cases, it may be beneficial to use automatic calibration to “precalibrate” the instrument by executing each setup to be used in an initialization routine or similar function. This causes run-time calibrations to occur before the programs executes.

|                                                     |  |                 |  |
|-----------------------------------------------------|--|-----------------|--|
| <div>&lt;</div>                                     |  | <div>&gt;</div> |  |
| <div>2015 Teradyne, Inc. All rights reserved.</div> |  |                 |  |

# awg6G\_theory.2.17

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

## AWG6G Theory of Operation : Operating Concepts : Waveform Start Methods

### Waveform Start Methods

An AWG6G output waveform can be started synchronously or asynchronously to an event. Synchronous starts use microcodes originating from an HSD pattern. This method allows precise control over when a waveform starts. For more information, see [AWG6G Pattern Control](#). If the exact start time of the waveform segment is not important, it can be started asynchronously using a VBT command. With this method, a waveform starts as soon as the you issue a start command to the instrument. The exact time at which the IG-XL issues the start command is somewhat variable and is therefore considered asynchronous. For more information, see the [Start VBT statement](#).

Once started, the instrument can run the waveform segment only once or continuously. You can also program a waveform to begin after any currently running waveform completes.

|                 |                 |
|-----------------|-----------------|
| <div>&lt;</div> | <div>&gt;</div> |
|-----------------|-----------------|

|  |                                          |  |
|--|------------------------------------------|--|
|  | 2015 Teradyne, Inc. All rights reserved. |  |
|--|------------------------------------------|--|